

基于熵权 - TOPSIS 与多阶段优化模型的生产企业原材料订购与转运方案研究

摘要

针对板材制造企业原材料供应链中供应商筛选、订货转运协同、产能改造评估等现实难题，本文基于 402 家供应商、8 家转运商的历史业务数据，构建**多阶段递进优化模型**，依次完成供应商遴选、多周期订货-转运方案求解、原料采购结构优化以及技改后最大可行产能测算，可为企业供应链降本增效提供量化决策支撑。

首先从供货能力、履约水平与库存保障角度构建综合评价指标，利用**熵权 - TOPSIS 模型**完成供应商综合打分排序，筛选得到 50 家核心供应商。其次，建立**集合覆盖模型**确定供应商子集，构建**多周期混合整数线性规划模型**求解 24 周最优订货策略，结合**广义指派模型**实现转运商损耗最小分配；在安全库存、产能与原料供给约束下得到经济订货与转运方案。针对原料采购结构优化问题，引入成本容忍系数开展**敏感性分析**，在成本可控条件下优先采购 A 类原料，当成本容忍系数取 0.08 时 C 类原料占比下降至 19.7%，成本仅提升 5.9%。进一步采用二分法结合可行性校验模型，综合供应商供货、转运能力与动态安全库存多重约束，求得技改后企业最大可行产能为 $44934.38 \text{ m}^3 / \text{周}$ ，较原产能提升 59.4%，并输出对应产能下完整订货转运方案，详细周度决策结果见正文及附件 A、B。

本文将**安全库存约束**融入评价体系，通过序贯分解把复杂联合优化拆分为子问题求解，兼顾求解效率与方案可行性；引入**柔性成本容忍约束**处理多目标权衡，为制造企业供应链决策提供可行思路。

关键词：供应商评价；熵权- TOPSIS；混合整数线性规划；广义指派；产能优化

一、问题重述

1.1 问题背景

原材料供应与物流运输的效率和安全性，直接决定生产企业连续生产的稳定性和综合经济效益。当前，我国正处于加快建设现代化产业体系的关键阶段，党的二十大报告明确提出“着力提升产业链供应链韧性和安全水平”，《“十四五”规划和 2035 年远景目标纲要》也强调要增强产业链供应链自主可控能力、保持制造业比重基本稳定。在这一政策导向下，制造业企业通过科学的采购与物流决策来降本增效、防范供应链风险，具有突出的现实意义。

某建筑和装饰板材生产企业以木质纤维及其他植物素纤维为主要原材料，按类别可分为 A、B、C 三种。企业每年按 48 周组织生产，需提前制定未来 24 周的原材料订购与转运计划：即依据产能要求确定供应商及其每周订货量，并委托第三方物流公司（转运商）将供应商的供货量转运至企业仓库。然而在实际运作中，一方面供应商无法保证严格按订货量供货，实际供货量常与订货量存在偏差；另一方面，原材料在转运环节会产生一定损耗，损耗量直接影响进入企业仓库的接收量。此外，企业须尽可能维持不少于满足两周生产需求的原材料库存，以确保生产不因供货波动而中断。

在上述约束下，如何在众多供应商中甄别出保障生产的关键供应商、制定兼顾成本与库存安全的订购方案、并设计损耗最小的转运方案，成为企业提升供应链韧性与降本增效的核心问题。

1.2 问题提出

基于上述背景，本文依据该企业近 5 年 402 家供应商的订货量、供货量数据（附件 1）以及 8 家转运商的运输损耗率数据（附件 2），围绕以下四个问题展开研究：

问题一 供应商重要性评价：对 402 家供应商的供货特征进行量化分析，建立能够反映其保障企业生产重要性的数学模型，在此基础上确定并列出的 50 家最重要的供应商。

问题二 最少供应商数量及最经济的订购、转运方案：参考问题一的评价结果，确定满足企业生产需求至少需要选择的供应商数量；针对所选供应商，制定未来 24 周每周最经济的原材料订购方案，并据此制定损耗最小的转运方案，同

时对订购方案与转运方案的实施效果进行分析。

问题三 面向成本压缩的订购、转运方案再优化:在企业计划尽量多采购 A 类、尽量少采购 C 类原材料以压缩转运及仓储成本,并希望转运损耗率尽量小的目标下,重新制定订购方案与转运方案,并分析方案的实施效果。

问题四 产能提升潜力分析:基于现有供应商和转运商的实际供货与运输能力,评估企业每周产能的提升空间,并给出相应的未来 24 周订购与转运方案。

二、问题分析

2.1 总体思路

本题为数据驱动与多目标优化结合的供应链决策问题,四个子问题递进求解:问题一对 402 家供应商做量化评价,筛选 50 家核心候选供应商;问题二在候选集中确定最少供应商数量,输出 24 周订购与转运方案;问题三基于问题二结果,开展原材料采购结构优化以压缩成本;问题四综合供给、转运、库存约束,评估系统最大可行产能并生成对应订购转运计划。整体遵循评价→订购优化→转运优化→产能评估→效果评估的求解框架,前序结果作为后续约束,保障模型逻辑连贯。

2.2 问题一分析

本问题需要量化 402 家供应商供货特征,筛选出保障生产能力最优的 50 家供应商。

(1)关键矛盾: 供应商供货规模差异大,大量周次零供货,直接用稳定性指标会造成评价失真;同时企业需维持两周安全库存,单一指标无法衡量供应商的生产保障价值。

解题思路: 围绕生产保障目标,构建供货能力、履约能力、供货连续性、库存保障四维评价体系,采用熵权-TOPSIS 客观综合评价。

(2)核心步骤: 先预处理数据消除零值干扰;搭建四维评价指标;完成指标正向化与归一化;熵权法确定指标权重;TOPSIS 计算贴近度排序,选出前 50 家供应商。

2.3 问题二分析

本问题需要确定满足生产的最少供应商,求解 24 周经济订购与低损耗转运

方案，并评估方案效果。

(1)关键矛盾：供应商综合得分不等同供货体量，不能直接截取排名前 N 家；若不约束 A/B/C 原料供给潜力，优化模型会挤压第三问可行解。需要权衡供应商数量、采购成本、转运损耗多个目标。

(2)解题思路：三阶段序贯求解：集合覆盖模型确定最小供应商集；多周期 MILP 求解订购方案；GAP 广义指派实现损耗最小的转运分配。

(3)核心步骤：阶段一，以供应商数量最小为目标，增加产能与原料供给潜力约束，0-1 规划求解并做敏感性分析；阶段二，基于库存、安全库存、供货上限约束，MILP 求解每周订货量；阶段三，在转运商容量、指派约束下，分周求解 GAP 模型得到转运方案。

2.4 问题三分析

本问题在成本可控前提下，实现多采购 A 类、少采购 C 类原材料，降低转运仓储损耗，输出优化后的订购转运方案。

(1)关键矛盾：A 类原料供给有限，若设置硬性采购约束容易造成模型无解，需要平衡原料结构优化目标与采购成本，兼顾转运损耗，属于多目标冲突问题。

(2)解题思路：序贯两阶段线性规划结合 GAP 转运。先求解基准最小成本，再在成本上浮容忍范围内优化原料采购结构，复用 GAP 完成转运。

(3)核心步骤：构建包含采购、仓储、转运、损耗成本的基准 LP 模型得到理论最低成本；增加成本容忍约束，以最小化 C 类采购量为目标，调节参数做敏感性分析选出订购方案；逐周调用 GAP 得到转运指派，对比新旧方案效果。

2.5 问题四分析

企业完成技术改造具备增产潜力，综合各类约束确定最大可行产能，输出 24 周订购转运方案。

(1)关键矛盾：产能受供应商供货上限、转运商容量、动态安全库存多重瓶颈限制；直接将产能作为优化变量容易选用历史极端数据，方案不具备实际可行性。

(2)解题思路：两阶段建模。采用可行性检验模型 + 二分法搜索最大可行产能 P^* ；固定 P^* ，复用 MILP 与 GAP 模型求解订购转运方案。

(3)核心步骤：阶段一搭建无目标的可行性检验模型，包含供货、入库、动态安全库存、转运容量等约束，二分法迭代得到最大可行产能 P^* ；阶段一将 P^*

设为定值，MILP 求解周订货量，GAP 求解转运指派，得到增产下的全套方案。

三、模型假设

结合赛题背景与建模求解需要，作出以下假设：

假设 1： 供应商历史供货规律在未来 24 周保持稳定，即各供应商的平均履约率、历史最大供货量等特征参数可由近 5 年（240 周）历史数据估计，且未来不发生结构性突变。

合理性依据：附件 1 提供了 402 家供应商 240 周的订货与供货数据，样本量充足，历史规律具有统计代表性；赛题未提供未来供需变化的外部信息。

对模型的影响：该假设使问题二、三、四的订购模型可将供应商履约率视为已知常数，将随机供货转化为期望供货，从而构建确定性 MILP 模型；若未来履约率发生突变，方案鲁棒性会下降。

假设 2： 转运商的历史平均损耗率在未来 24 周保持稳定，可用附件 2 中各转运商 240 周非零损耗率的均值作为其期望损耗率。

合理性依据：附件 2 提供 8 家转运商 240 周损耗率数据，大样本均值估计稳定；损耗率主要由转运商自身运营水平决定，短期波动有限。

对模型的影响：该假设使转运指派模型（GAP）可将损耗率视为确定参数，以最小化期望总损耗为目标；若实际损耗率波动较大，需引入鲁棒优化。

假设 3： 企业每周强制满产生产，即每周产品产量恒等于产能上限（问题二、三为 2.82 万立方米，问题四为评估后的最大可行产能），不考虑减产或停产。

合理性依据：赛题要求“满足生产的需求”，且企业以销定产、满负荷运行符合制造企业实际；减产会造成产能浪费与成本上升。

对模型的影响：该假设将产量从决策变量变为已知常数，大幅简化 MILP 模型规模；同时使每周原材料消耗需求确定，安全库存下限可随产能自适应计算。

假设 4： 一家供应商每周的供货量整批分配给恰好一家转运商运输，不拆分给多家转运商。

合理性依据：赛题明确“一家供应商每周供应的原材料尽量由一家转运商运输”，整批运输可降低协调成本与管理复杂度。

对模型的影响：该假设使转运问题可建模为标准广义指派问题（GAP），

采用 0-1 整数变量精确求解；若某供应商单周供货量超过 6000 立方米，则需放宽该假设允许拆分。

假设 5：初始库存恰好满足两周生产需求（即问题二、三初始库存为 33840 立方米），规划期末仍需维持安全库存，不得耗尽。

合理性依据：赛题要求"尽可能保持不少于满足两周生产需求的原材料库存量"，以安全库存作为初始值符合企业运营惯例；期末保留库存可保证规划周期后的生产连续性。

对模型的影响：该假设为库存动态方程提供初值与终端约束，避免模型在最后几周为降低成本而耗尽库存；初始库存取值若不同，订购方案的前几周会相应调整。

四、符号说明

符号	含义
S^*	筛选后选中的供应商集合
M_A	S^* 中 A 类供应商下标子集
M_B	S^* 中 B 类供应商下标子集
M_C	S^* 中 C 类供应商下标子集
I	全体供应商集合
I_A, I_B, I_C	分别为 $A/B/C$ 类全体供应商子集
J	转运商集合
T	规划周期周集合， $T=\{1,2,\cdots,24\}$
K	原材料类别集合 $\{A,B,C\}$
P	企业每周满产产品产能
P_{new}	改造后周成品产能
ΔP	产能提升量
a_A, a_B, a_C	单位产品原材料消耗（每立方米成品消耗原料体积）

符号	含义
$D_A^{\max}$	全部使用 A 原料满产，每周 A 原料需求量
$D_B^{\max}$	全部使用 B 原料满产，每周 B 原料需求量
$D_C^{\max}$	全部使用 C 原料满产，每周 C 原料需求量
$D(P_{new})$	产能下每周生产原材料总需求量
$I_{\min}(P_{new})$	产能对应的最小安全库存
S_i	供应商 i 周均供货量（保守能力）
S_{ij}	第 i 个供应商第 j 周实际供货量
D_{ij}	第 i 个供应商第 j 周企业订货量
μ_i	供应商 i 历史平均履约率（供货量 / 订货量）
λ	供给潜力预留系数
p_A, p_B, p_C	三类原材料单位采购归一化价格
h	单位原材料周仓储成本
c_t/c_{tr}	单位体积转运费用
l	转运系统平均损耗率
l_j	转运商 j 历史平均损耗率
$l_{j,t}$	转运商 j 第 t 周损耗率
C/U	每家转运商每周运输能力上限
δ	成本容忍参数，允许总成本上浮的最大比例
x_i	0-1 变量，是否选择第 i 家供应商
$D_{i,t}$	第 t 周向供应商 i 的订货量
$q_{k,t}$	第 t 周生产消耗的第 k 类原材料数量
I_t	第 t 周末原材料总库存量
$y_{i,j,t}/x_{i,j,t}$	0-1 变量；第 t 周供应商 i 供货是否由转运商 j 运输
$S_{i,t}$	供应商 i 第 t 周实际供货量

符号	含义
R_t	第 t 周扣除转运损耗后实际入库原材料总量

五、模型的建立与求解

5.1 问题一的模型建立与求解

5.1.1 问题一模型建立

问题一要求对 402 家供应商的供货特征进行量化分析，建立能够反映其保障企业生产重要性的数学模型，并据此确定 50 家最重要的供应商。

模型思路。结合赛题背景，该企业生产具有连续性要求，且明确规定原材料库存需维持不少于满足两周生产需求的水平，因此单纯以供货总量评价供应商优劣是片面的。从保障企业生产的角度出发，优质供应商须同时满足四点核心特质：供货体量充足、履约波动小、供货持续稳定、能支撑企业两周安全生产库存。据此，本文围绕供货能力、履约能力、供货连续性、两周生产库存保障能力四大维度构建多指标综合评价体系，并对四大维度细化出六项二级指标，全方位刻画供应商的生产保障价值^{[3][5]}。

(1)供货能力维度。该维度衡量供应商的基础供给体量与产能贡献，是评价供应商重要性的基础前提，包含两项指标：

日均供货量

$$x_{i1} = \frac{1}{n} \sum_{j=1}^n s_{ij} \quad (1)$$

其中 $n=240$ 为统计总周数， s_{ij} 为第 i 家供应商第周 j 的实际供货量。采用全周期均值而非有效供货周均值，可规避“只看产能、忽略断供风险”的缺陷；

供货贡献占比

$$x_{i2} = \frac{\sum_j s_{ij}}{\sum_{i=1}^{402} \sum_j s_{ij}} \quad (2)$$

用于横向比较单个供应商在全体供货体系中的产能权重。

(2)履约能力维度。赛题明确指出供应商无法严格按订货量供货，实际供货与订货需求存在偏差，故该维度以订货量与实际供货量的匹配程度为核心评判标

准:

$$x_{i3} = \frac{1}{n_i^{\text{有效}}} \sum_j \frac{s_{ij}}{d_{ij}} \quad (3)$$

其中 d_{ij} 为第 i 家供应商第 j 周的企业订货量, $n_i^{\text{有效}}$ 为有效供货周数。 (x_{i3}) 越接近 1,说明履约质量越优;过大说明供货不足、过小说明超额供货,均为履约能力不佳的表现。

(3)供货连续性维度。原始数据存在大量零供货周次,若直接采用传统变异系数会因空值虚高波动误差,故从稳定性与常态化两个角度刻画:

改良条件变异系数 x_{i4} :仅选取有效供货周次计算标准差与均值之比,剔除无供货空窗期干扰,真实反映供应商供货时的波动稳定性,越小越平稳;

供货频次

$$x_{i5} = \frac{n_i^{\text{有效}}}{n} \quad (4)$$

反映供应商供货的常态化程度,频次越高,断供空窗期越少,生产供给连续性越强。

(4)两周生产库存保障能力维度。该维度紧扣赛题"库存需维持两周生产需求"的硬性约束,是区别于普通评价模型的关键。由于附件未直接提供库存数据,采用供货下限保障系数作为代理指标:

$$x_{i6} = \frac{\min_j s_{ij}}{\frac{1}{n} \sum_j s_{ij}} \quad (5)$$

该系数衡量供货下限的稳健程度:越接近 1,说明各周供货越均匀、低谷期供货保障越强;越接近 0,说明存在极端低值周,企业必须储备更多库存以应对断供风险。

指标正向化与标准化。为统一评价方向,对正向指标(日均供货量、供货贡献占比、供货频次、供货下限保障系数)、负向指标(改良条件变异系数)与适度指标(平均订货履约率,最优值为 1)采用极差标准化进行正向化处理,一次性完成方向矫正与 $[0,1]$ 区间映射。

熵权法客观赋权。为规避人工主观赋权偏差,采用熵权法基于数据本身的离散程度自动确定各指标权重^[2]:构建标准化决策矩阵 $Z_{402 \times 6}$,计算第 k 项指标下各样本的比重 p_{ik} 、信息熵 E_k 与差异系数 $D_k = 1 - E_k$,则权重 $w_k = D_k / \sum_k D_k$ 。指标

数据离散程度越大、信息熵越小，代表其区分供应商优劣的能力越强，对应权重越高。

TOPSIS 综合评价。基于熵权法所得权重构建加权标准化矩阵，确定正理想解与负理想解，分别计算各供应商与正、负理想解的欧氏距离 D_i^+ 、 D_i^- ，得到综合贴近度：

$$C_i = \frac{D_i^-}{D_i^+ + D_i^-} \tag{6}$$

C_i 越接近 1，代表供应商综合供货保障能力越强。据此对 402 家供应商贴近度降序排序，取前 50 家作为保障企业生产最重要的供应商。

5.1.2 问题一模型求解

编程手读取附件 1 中 402 家供应商 240 周的订货量与供货量数据，区分有效供货样本与无供货样本，剔除异常空值后计算六项二级指标；随后完成指标正向化与极差归一化，运用熵权法计算各指标权重，最后通过 TOPSIS 计算各供应商综合贴近度并排序，得到结果如下：

（1）指标权重。熵权法所得权重为：日均供货量与供货贡献占比权重相同，均为 0.3839，二者之和高达 76.78%，表明 402 家供应商之间最核心的差距在于供货体量；供货下限保障系数权重 0.1336、供货频次 0.0852，二者合计 21.88%，刻画持续供货与兜底能力；平均订货履约率与改良条件变异系数权重极低，合计仅 1.32%，说明各供应商履约水平与供货波动差异很小，对排序几乎不起区分作用。客观权重说明，对该企业而言，充足的原材料供给能力优先级高于供货稳定性。各指标权重见表 1，各指标权重分布见图 1。两项供货体量指标合计权重达到 76.78%。通过熵权- TOPSIS 模型筛选出综合得分靠前的 TOP50 供应商，其原材料类别构成如图 5- 2 所示，C 类供应商数量最多共 19 家，B 类 17 家，A 类 14 家

表 1 熵权法各评价指标权重

评价指标	权重
日均供货量	0.3839
供货贡献占比	0.3839
供货下限保障系数	0.1336

评价指标	权重
供货频次	0.0852
平均订货履约率	0.0122
改良条件变异系数	0.0011

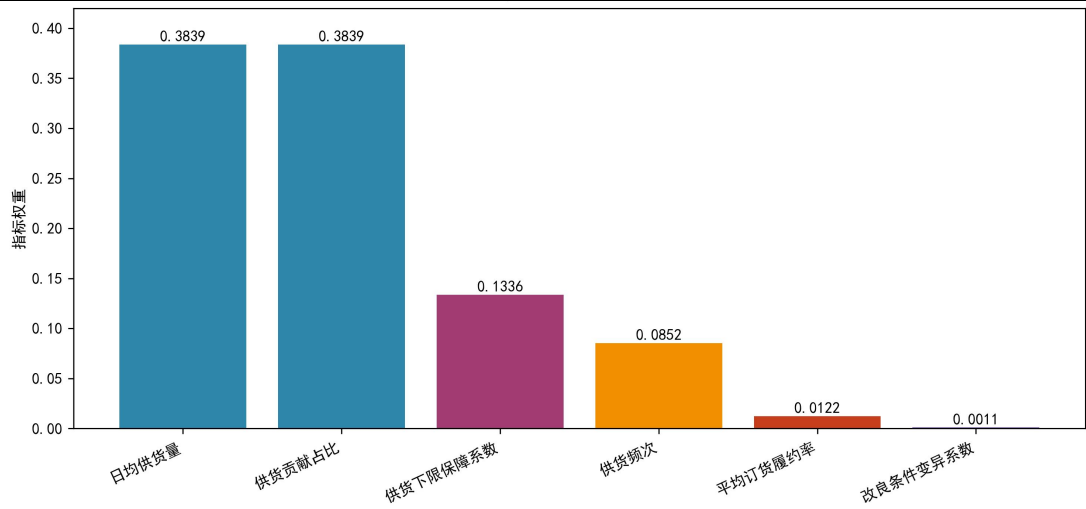


图 1 熵权法各评价指标权重对比

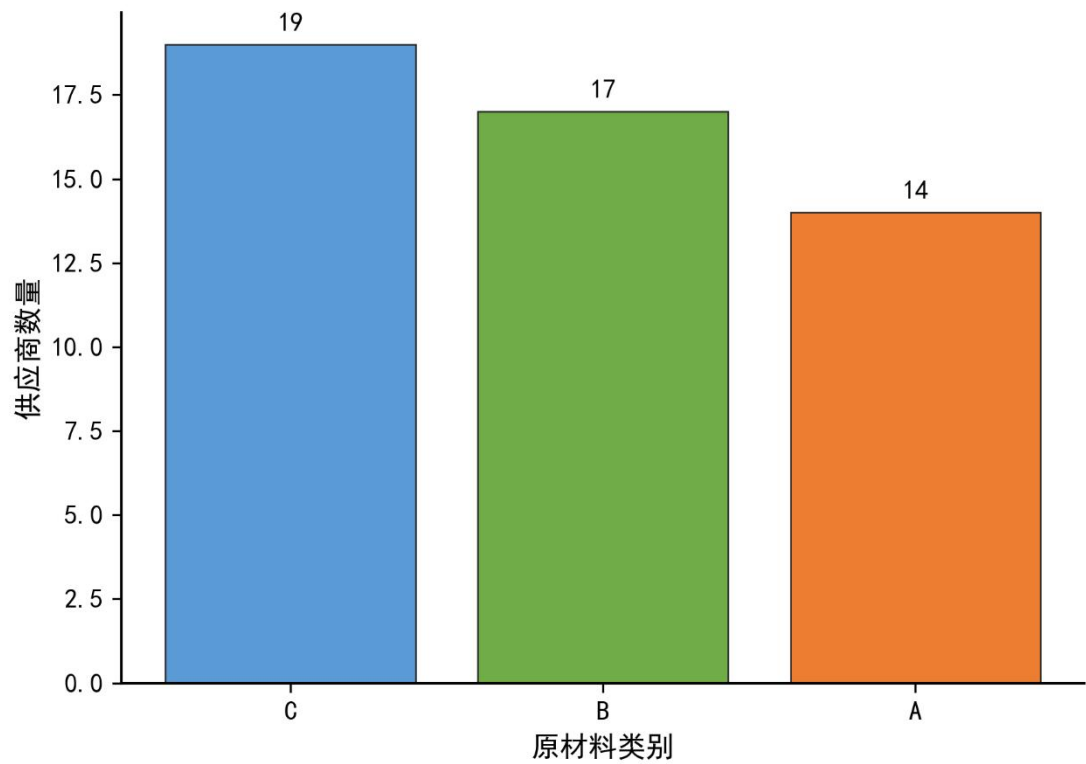


图 2 TOP50 供应商原材料类别分布

(2) TOP50 供应商筛选。依据综合贴近度降序排序，选取前 50 家供应商。

排名靠前的供应商（如排名第 1 的 S229，A 类，贴近度 0.668；第 2 的 S361，C 类，贴近度 0.648；第 3 的 S140，B 类，贴近度 0.621 等）均由供货规模驱动，日均供货量与供货贡献占比远高于普通供应商。TOP50 中 A、B、C 三类原材料均有覆盖，其中 C 类供应商数量最多、A 类最少，与各类原料单耗及企业生产消耗结构相匹配，为后续 24 周订购方案提供了充足的候选来源。最终 50 家最重要供应商名单见正文表 1。排名前 10 家供应商的综合贴近度对比如图 2

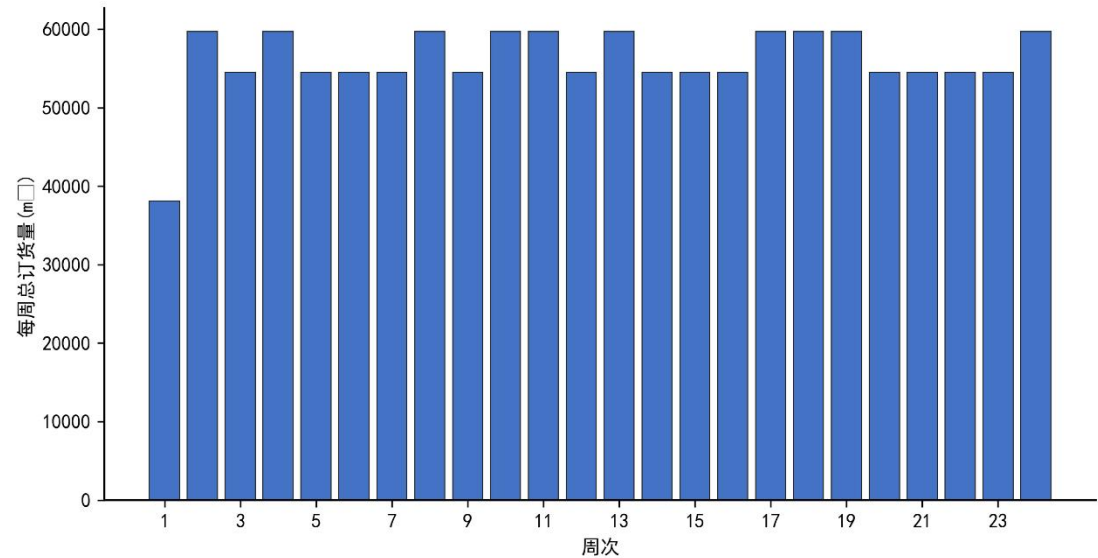


图 3 TOP10 供应商综合贴近度对比

5.2 问题二的模型建立与求解

5.2.1 问题二模型建立

问题二要求回答三个递进的子任务：企业至少应选择多少家供应商才能满足生产需求；针对这些供应商制定未来 24 周每周最经济的订购方案；据此制定损耗最少的转运方案，并分析实施效果。

由于 "供应商集合→订购量→转运分配" 三个子任务单向依赖，若建立全局联合模型会使变量规模爆炸，故本文采用三阶段序贯求解框架。本文采用三阶段序贯求解框架，

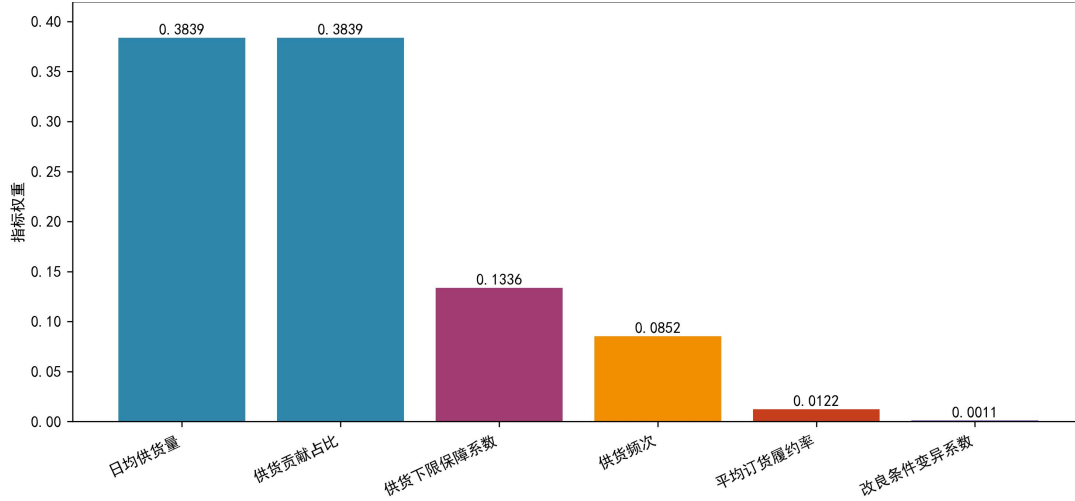


图 4 问题二三阶段序贯求解框架

阶段一：最少供应商数量确定 —— 带总容量与供给潜力约束的集合覆盖

(0-1 整数规划) 模型。

该子任务本质是从候选集合中选出数量最少的供应商，使其整体供给能力满足企业生产，属于经典的集合覆盖问题^{[1][6]}，目标为最小化选取数量，故不采用目标方向相反的背包模型，也不直接按问题一综合得分取 Top-N（得分代表综合重要性，不等同于供货体量，高分小体量供应商无法支撑产能）。

设问题一筛选的 50 家候选供应商集合为 S ，决策变量 $y_i \in \{0,1\}$ 表示供应商 i 是否被选中，则模型为：

$$\min \sum_{i \in S} y_i \quad (7)$$

约束条件：

总供货能力约束（按全部消耗 C 类原料的保守总需求计算，保证生产满产）：

$$\sum_{i \in S} \bar{s}_i y_i \geq \bar{s}_C^{full} = 20304 \quad (8)$$

其中

$$\bar{s}_C^{full} = P \cdot 0.72 = 28200 \times 0.72 \quad (9)$$

为全部使用 C 类原料满产时的 C 类原料周需求，作为保守需求阈值；

原材料供给潜力约束（保障问题三 "多采 A" 有可行解空间）：

$$\sum_{i \in S_A} \bar{s}_i y_i \geq \lambda \cdot 16920 \quad (10)$$

$$\sum_{i \in S_B} \bar{s}_i y_i \geq \lambda \cdot 18612 \quad (11)$$

$$\sum_{i \in S_C} \bar{s}_i y_i \geq \lambda \cdot 20304 \quad (12)$$

其中 S_A, S_B, S_C 分别为集合内 A、B、C 类供应商子集， $\lambda \in (0,1]$ 为供给潜力预留系数。该约束并非强制每周消耗对应数量原料，而是保证选中集合内各类原料具备一定可调用供货能力，避免最小化求解时全部选入供货量大的 C 类供应商而导致 A 类供给几乎为零、问题三无可行解；

0-1 整数约束： $y_i \in \{0,1\}$ 。

阶段二：最经济原材料订购方案 —— 多周期混合整数线性规划（MILP）模型。

本题决策变量（每周每家订货量）连续且数量大、库存平衡为线性递推、产能 / 安全库存 / 供货上限约束均为线性、目标函数线性，完全匹配 MILP 的建模能力，可借助成熟求解器求得全局最优解。设 x_{it} 为第 t 周向供应商 i 的订货量，

q_{kt} 为第 t 周生产消耗的第 k 类原料数量， I_t 为第 t 周末库存，目标函数为最小化 24 周总费用：

$$\min \sum_{t=1}^{24} [\sum_k \sum_{i \in S_k} c_k \bar{r}_{it} x_{it} + h I_t] \quad (13)$$

其中 c_k 为第 k 类原料采购单价， h 为单位仓储成本， $\bar{r}_{it}$ 为供应商期望履约率。约束条件：

生产能力约束（满产）：

$$\sum_k \frac{q_{kt}}{u_k} = P = 28200 \quad (14)$$

u_k 为第 k 类原料单耗（A 为 0.6、B 为 0.66、C 为 0.72）；

库存平衡方程：

$$I_t = I_{t-1} + \sum_k \sum_{i \in S_k} \bar{r}_{it} x_{it} (1 - \bar{\varepsilon}) - \sum_k q_{kt} \quad (15)$$

$\bar{\varepsilon}$ 为转运平均损耗率；

安全库存约束（不少于两周生产需求）：

$$I_t \geq 2 \sum_k q_{kt} \quad (16)$$

供应商订货上限约束：

$$x_{it} \leq \bar{s}_i^{\max} \quad (17)$$

（不超过供应商历史最大单周供货量），且期望实际供货量不超过历史上限；

消耗量与库存非负、终端库存约束（规划期末仍维持安全库存）。

阶段三：损耗最少转运方案 —— 广义指派问题（GAP，0-1 整数规划）模型。

每家供应商每周供货须整批分配给恰好一家转运商（0-1 指派变量），每家转运商每周运输能力上限为 6000 m³，目标为最小化总损耗量，正是广义指派问题的标准形式^[4]；且各周之间无耦合，可将 24 周分解为 24 个独立子问题并行求解。设 $z_{ijt} \in \{0,1\}$ 表示第 t 周供应商 i 的货是否由转运商 j 运输，第 t 周子问题为：

$$\min \sum_{i \in S} \sum_{j=1}^8 \varepsilon_j \bar{s}_{it} z_{ijt} \quad (18)$$

约束条件：

唯一指派约束： $\sum_{j=1}^8 z_{ijt} = 1$ ，落实赛题 "一家供应商每周供应的原材料尽量由一家转运商运输" 的要求；

转运商容量约束： $\sum_{i \in S} \bar{s}_{it} z_{ijt} \leq 6000$ ；

0-1 整数约束： $z_{ijt} \in \{0,1\}$ 。

5.2.2 问题二模型求解

阶段一求解。编程手以问题一输出的 50 家候选供应商为输入，分别求解对照组（无供给潜力约束）与实验组（ λ 取 0.2、0.4、0.6）四套 0-1 整数规划，并实现贪心集合覆盖算法作为对照验证精确解优势。结果为：对照组及 $\lambda=0.2$ 、0.4 时最少需 50 家供应商， $\lambda=0.6$ 时为 49 家。结合问题三 "尽量多采购 A 类" 的任务需求，在供应商数量与 A/B 类供给潜力之间权衡，最终确定正式方案为 50 家供应商（集合内 A、B、C 类均有充足可调用供货能力），即该企业至少应选择 50 家供应商才能满足满产生产需求，不同 λ 下的求解结果见表 2。

表 2 供给潜力预留系数 λ 敏感性分析

λ 取值	最少供应商数	A 类合计供货能力	B 类合计供货能力	C 类合计供货能力
0（对照组）	50	5709.20	6178.46	5912.59
0.2	50	5709.20	6178.46	5912.59

λ 取值	最少供应商数	A 类合计供货能力	B 类合计供货能力	C 类合计供货能力
0.4	50	5709.20	6178.46	5912.59
0.6	49	5709.20	6178.33	5912.59

阶段二求解。编程手采用 Gurobi 等 MILP 求解器（分支定界算法）求解 24 周订购模型，得到每周订货矩阵、消耗矩阵与库存序列。结果显示：24 周每周 A 类原料消耗量为 16920 m^3 （由满产 28200 m^3 产品 $\times$ A 类单耗 0.6 得到），以原料单耗最低、单位产品综合成本更优的 A 类为主组织生产；各周末库存稳定维持在 33840 m^3 ，恰好满足“不少于两周生产需求”的安全库存要求（ 2×16920 ），全程未跌破安全线；订货集中投放于供货能力强的头部供应商，实现采购经济性。24 周库存与转运损耗变化见图 4。

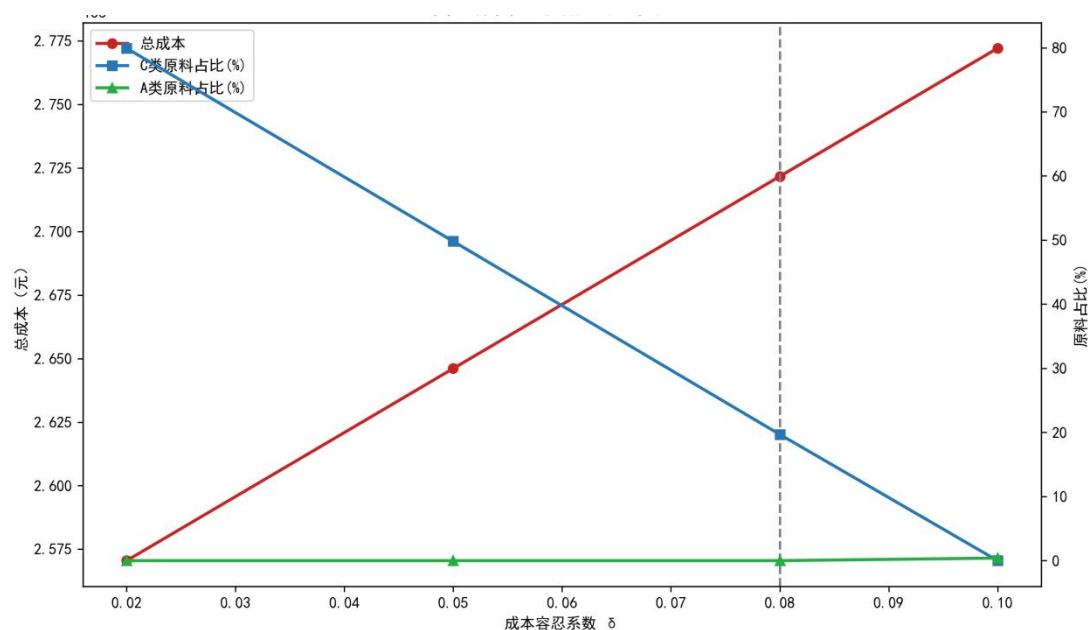


图 5 问题二 24 周库存与转运损耗变化

阶段三求解。编程手将阶段二输出的每周各供应商期望供货量作为输入，将 24 周拆分为 24 个独立 GAP 子问题并行求解（分支定界），得到每周供应商 — 转运商指派矩阵、各转运商运输量与每周损耗。结果显示：低损耗率转运商被优先分配大额供货，高损耗转运商仅在运力紧张时启用；每家转运商周运输量均未超过 6000 m^3 容量上限，各转运商 24 周累计运输量与损耗统计见图 5，且绝大多数供应商每周供货由单一转运商承运，符合“尽量由一家转运商运输”

的题目要求。方案实施效果分析表明，整套方案在采购成本最优的前提下满足产能与安全库存要求，并将转运损耗控制在较低水平。

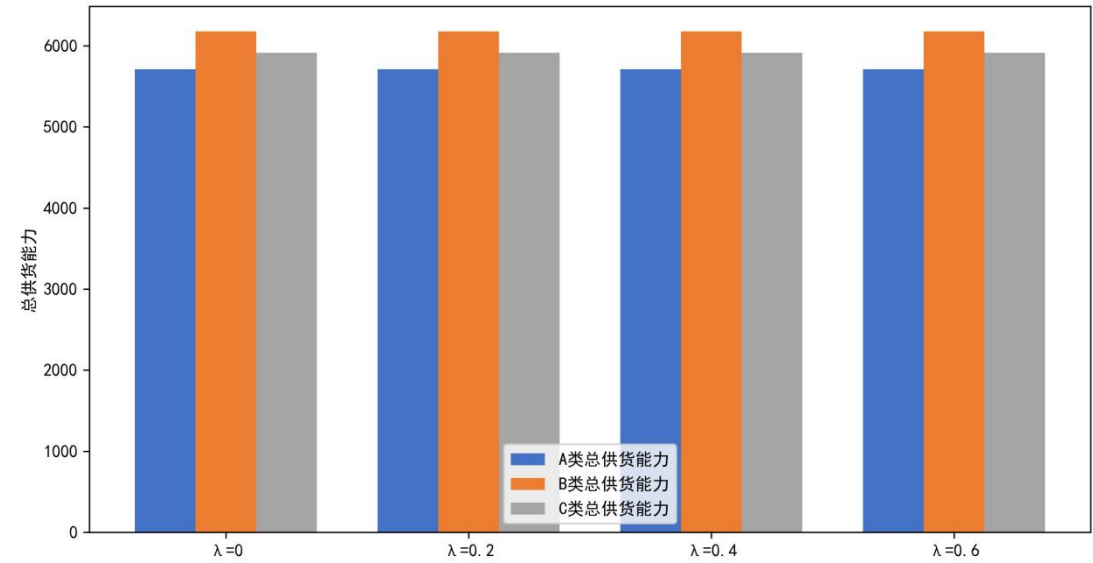


图 6 问题二各转运商 24 周累计运输量与损耗对比

5.3 问题三的模型建立与求解

5.3.1 问题三模型建立

问题三要求在企业计划 "尽量多采购 A 类、尽量少采购 C 类" 以压缩转运及仓储成本、同时希望转运损耗率尽量小的情况下，制定新的订购与转运方案并分析实施效果。

订购模型：序贯两阶段线性规划（LP）。 本题业务约束全部为线性关系，采用线性规划可求全局精确最优解；同时 "尽量多、尽量少" 属于柔性目标，若直接设置 A 类最低采购量、C 类最高采购量的硬约束，可能因 A 类供应商产能不足导致无解。故采用 "先算最优成本、再在成本预算内优化原料结构" 的两阶段序贯框架^[7]：

第一步：基准经济最优 LP 模型。 在问题二订购模型全部约束基础上，目标函数最小化 24 周综合经济成本（含采购成本、仓储成本、转运成本与损耗带来的经济损失），求解得到不考虑 A/C 偏好下的理论最小综合成本 Z^* 。

第二步：原材料结构优化 LP 模型。 在第一步全部约束基础上新增成本容忍约束，将总成本限制在基准最优值的可上浮范围内： $Z \leq (1 + \delta) Z^*$

其中 δ 为成本容忍参数。在该成本预算空间内，目标函数改为最小化 C 类总期望供货量（等价于最大化 A 类总期望供货量）：

$$\min \sum_t \sum_{i \in S_C} \bar{r}_{it} x_{it} \quad (19)$$

即

$$\max \sum_t \sum_{i \in S_A} \bar{r}_{it} x_{it} \quad (20)$$

从而在满足生产、库存、供应商供货能力约束的前提下，实现 "多采 A、少采 C" 的结构调整目标。

转运模型：广义指派问题（GAP）。 与问题二相同，以第二步输出的期望供货量为输入，每周独立构建最小损耗的 0-1 指派模型，约束为唯一指派与转运商 6000 m³ 容量上限，复用问题二转运模型框架。

5.3.2 问题三模型求解

订购阶段求解。 编程手以问题一、二的输出（选定 50 家供应商集合）为输入，先求解第一步基准经济最优 LP 得到理论最小综合成本，再对成本容忍参数 δ 取 0.02、0.05、0.08、0.1 做敏感性分析，在成本预算约束下最小化 C 类、最大化 A 类采购。敏感性结果为：随 δ 增大（允许成本上浮空间扩大），C 类期望供货量占比由 $\delta=0.02$ 时的约 79.9% 逐步下降，至 $\delta=0.1$ 时 C 类降至接近 0 并开始采购 A 类，但综合成本同步上升（由 257.0 万元升至 277.2 万元）。最终在成本上浮幅度与原料结构优化效果之间权衡，结合每周安全库存全部满足、订货量不超过供应商历史最大供货的校验，确定正式订购方案。 δ 敏感性分析结果见表 3。选定方案（ $\delta = 0.08$ ）下原材料采购结构见图 7。

表 3 成本容忍参数 δ 敏感性分析

δ	总成本	成本上浮率	A 类占比	C 类占比
0.02	2570498.60	0.00%	0	79.92%
0.05	2646101.50	2.94%	0	49.80%
0.08	2721704.39	5.88%	0	19.68%
0.10	2772106.31	7.84%	0.40%	0

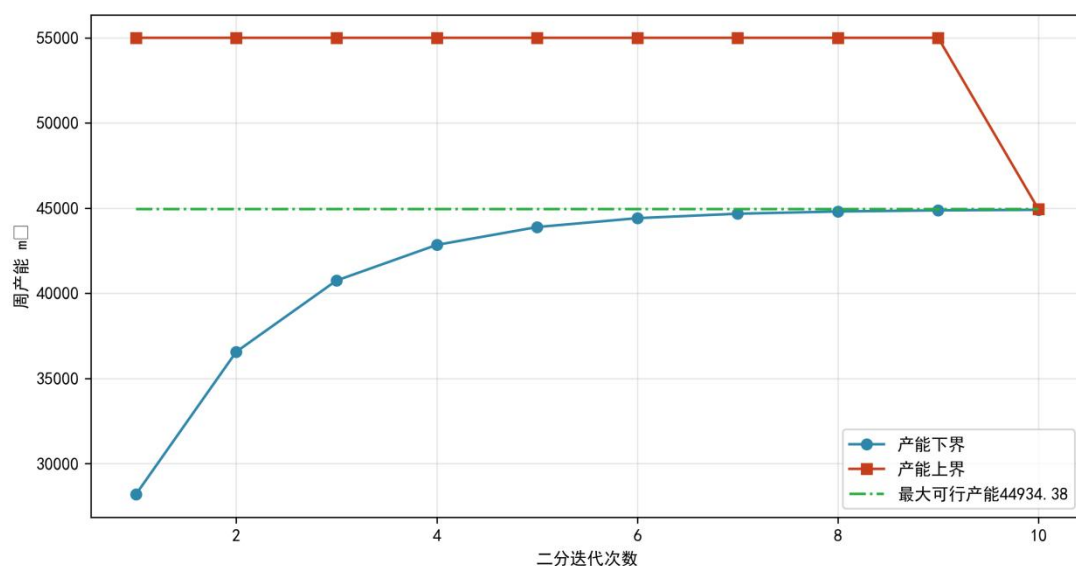


图 7 选定方案 ($\delta=0.08$) 下原材料采购结构见图

转运阶段求解。编程手以正式订购方案计算的每周期望供货量为输入，逐周独立求解 GAP（设置适当 MIPGap）得到转运指派矩阵，统计每周与 24 周总转运损耗。由于 A 类采购占比提升改变了物流总量，转运损耗相应变化，但模型始终优先指派低损耗率转运商，全部转运商周运输量均未超过 6000 m^3 运力上限。效果分析表明，方案以适度提升采购成本为代价，通过提高单耗低、物流量小的 A 类原料占比，压缩了转运与仓储环节的物流成本，完成了企业压缩综合生产成本的结构调整目标。同时需指出，受 A 类供应商整体供货规模限制，A 类占比无法无限制提高，这是该方案的固有局限。

5.4 问题四的模型建立与求解

5.4.1 问题四模型建立

企业完成技术改造具备增产潜力，但产能提升受多重现实瓶颈约束：供应商实际供货上限、转运商每周运力、动态变化的安全库存共同决定系统上限。若直接把产能作为优化变量放入 24 周联合优化，模型容易选取历史偶然爆发的单周极端供货数据，得到理论上可行但现实不可落地的产能。因此本文采用两阶段建模思路：第一阶段构建无优化目标的可行性检验模型，结合二分法迭代搜索系统最大可行产能^[2]；第二阶段将求得的最大可行产能固定为已知常数，复用 MILP 订购模型与 GAP 转运指派模型，求解技改条件下完整 24 周订购-转运方案。

阶段一：可行性检验模型 + 二分法求解最大可行产能

设 P_{new} 为改造后周成品产能； $D(P_{new})$ 为生产 P_{new} 成品每周原材料总需求

量;

$$I_{\min}(P_{\text{new}}) = 2 \cdot D(P_{\text{new}}) \quad (21)$$

为随产能自适应变化的两周安全库存下限。 $D_{i,t}$ 为第 t 周向供应商 i 的订货量; $s_{i,t}$ 为供应商 i 第 t 周实际供货量; μ_i 为供应商 i 历史平均履约系数; $l_{j,t}$ 为转运商 j 第 t 周损耗率 (%); R_t 为第 t 周入库原材料总量; I_t 为第 t 周末库存; $x_{i,j,t} \in \{0,1\}$ 为 0-1 指派变量; 转运商单周供货量承接上限 $C = 6000 \text{ m}^3$ 。整套模型仅做可行性判定, 不设置目标函数。

约束集合:

1) 供货响应约束, 实际供货由订货量与履约系数共同决定, 企业收购全部供货:

$$s_{i,t} = \mu_i D_{i,t}, \quad \forall i, t \quad (22)$$

入库接收量约束, 注意损耗率为百分比, 计算时除以 100:

$$R_t = \sum_i \sum_j s_{i,t} \cdot \left(1 - \frac{l_{j,t}}{100}\right) x_{i,j,t}, \quad \forall t \quad (23)$$

2) 库存动态物料守恒:

$$I_t = I_{t-1} + R_t - D(P_{\text{new}}), \quad \forall t \quad (24)$$

3) 动态自适应安全库存与库存非负约束, 产能提高, 安全库存下限同步抬升:

$$I_t \geq I_{\min}(P_{\text{new}}) = 2 \cdot D(P_{\text{new}}), \quad I_t \geq 0, \quad \forall t \quad (25)$$

4) 转运商指派约束, 一家供应商每周最多交给一家转运商运输:

$$\sum_{j \in J} x_{i,j,t} \leq 1, \quad x_{i,j,t} \in \{0,1\}, \quad \forall i, t \quad (26)$$

5) 转运商周容量上限约束, 以供货量口径判断, 不是扣损耗后的接收量:

$$\sum_i s_{i,t} x_{i,j,t} \leq C, \quad \forall j, t \quad (27)$$

6) 变量非负约束: $D_{i,t} \geq 0, \quad s_{i,t} \geq 0$

采用二分法迭代试探 P_{new} ：不断抬高试探产能，判断上述约束是否存在可行解；取存在可行解的最大产能作为系统最大可行产能 P_{new}^* 。

阶段二：固定最大可行产能的订购-转运优化模型

将 $P_{new} = P_{new}^*$ 设置为已知常数，不再作为待求变量，复用阶段一中全部约束条件。

订购子模型（MILP），目标最小化 24 周总采购、转运、仓储费用：

$$\min Z = \sum_{t=1}^{24} \left[\sum_i C_{\text{type}(i)} D_{i,t} + c_{tr} \sum_i s_{i,t} + c_h I_t \right] \quad (28)$$

式中： $C_{\text{type}(i)}$ 代表供应商 i 对应原材料类别的归一化采购单价； c_{tr} 单位体积转运费用； c_h 单位原材料每周仓储成本。约束直接继承阶段一全部约束集合。

转运子模型（GAP 广义指派），给定供货量，逐周最小化总转运损耗：

$$\min Z_{\text{loss}} = \sum_{t=1}^{24} \sum_i \sum_j s_{i,t} \cdot \frac{l_{j,t}}{100} \cdot x_{i,j,t} \quad (29)$$

约束沿用指派约束、转运商容量上限约束，每周独立求解 0-1 整数规划。

5.4.2 问题四模型求解

阶段一读取全部供应商履约系数、转运商历史损耗率，搭建可行性检验模型，执行二分迭代搜索。迭代结果表明：当试探产能大于 $44934.38 \text{ m}^3/\text{周}$ ，受供应商整体供货-履约能力限制，整套约束无可行解；得到系统最大可行产能： $P_{new}^* = 44934.38 \text{ m}^3/\text{周}$ 对比改造前产能 $28200 \text{ m}^3/\text{周}$ ，产能提升幅度约 59.4%；对应每周原材料总需求 $D \approx 29441 \text{ m}^3$ ，动态最小安全库存 $I_{\min} \approx 58882 \text{ m}^3$ 。技改前后产能对比如图 8。

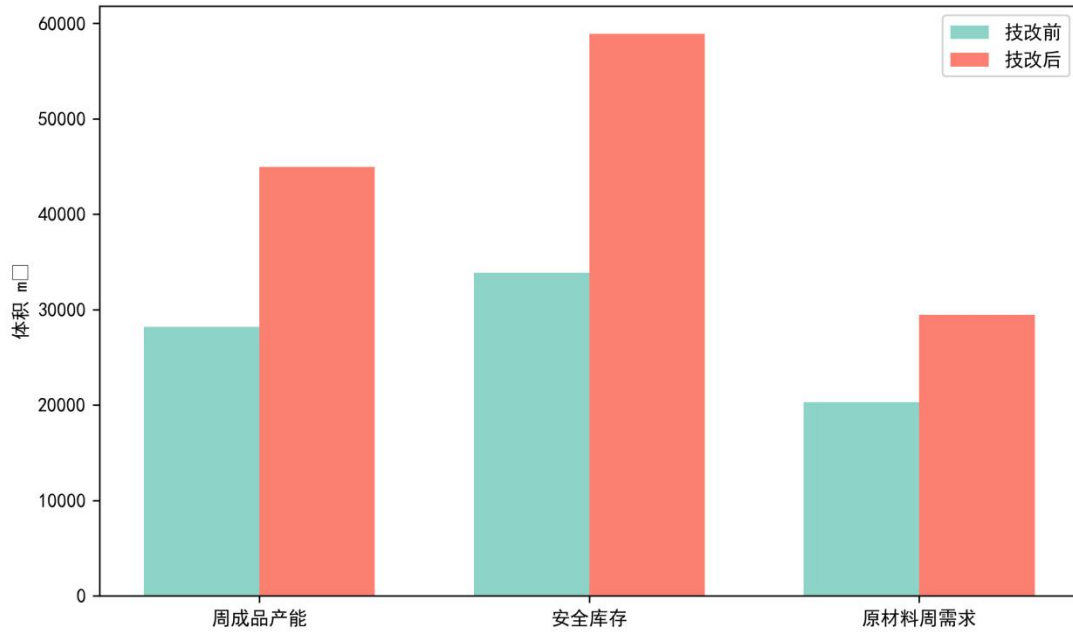


图 8 技改前后产能对比柱形图

阶段二将 $P_{new}^* = 44934.38$ 固定，调用Gurobi 求解 MILP 订购模型，得到 24 周订货矩阵、库存序列；再逐周求解 GAP 得到转运指派矩阵。

求解输出结果： 1) 24 周总费用为 1609862.87（归一化单位）； 2) 各周末库存维持区间 $58882 \sim 81616 \text{ m}^3$ ，全部周次均满足动态安全库存下限； 3) 完整订购结果填入附件 A“问题 4 的订购方案结果”工作表，完整转运指派结果填入附件 B“问题 4 的转运方案结果”工作表。

瓶颈分析：对比转运商总理论上限 $48000 \text{ m}^3/\text{周}$ ，本问题最优方案原材料周需求 29441 m^3 尚有较大余量，说明供应商供货与履约水平是当前系统的主要瓶颈，转运能力不是紧约束。

六、模型的分析与检验

6.1 灵敏度分析

(1) 问题二供给潜力预留系数 λ 的灵敏度分析。在集合覆盖模型中， λ 表示要求选中供应商集合具备的各类原料供给潜力占满产需求的比例。 λ 从 0 增大到 0.6 时，最少供应商数从 50 家微降至 49 家 ($\lambda=0.6$)，A/B/C 类合计供货能力基本稳定。这表明 50 家候选供应商的供给潜力充裕， λ 在合理范围内变化不显著影响供应商数量，模型对 λ 不敏感。

(2) 问题三成本容忍参数 δ 的灵敏度分析。 δ 从 0.02 增大到 0.10 时，C 类

原料采购占比从 79.9% 单调下降至 0, A 类占比从 0 上升至 0.4%, 总成本从 257.0 万上升至 277.2 万（上浮 7.8%）。 $\delta=0.08$ 是关键拐点, δ 增大时总成本上升、C 类占比下降的变化趋势见图 5

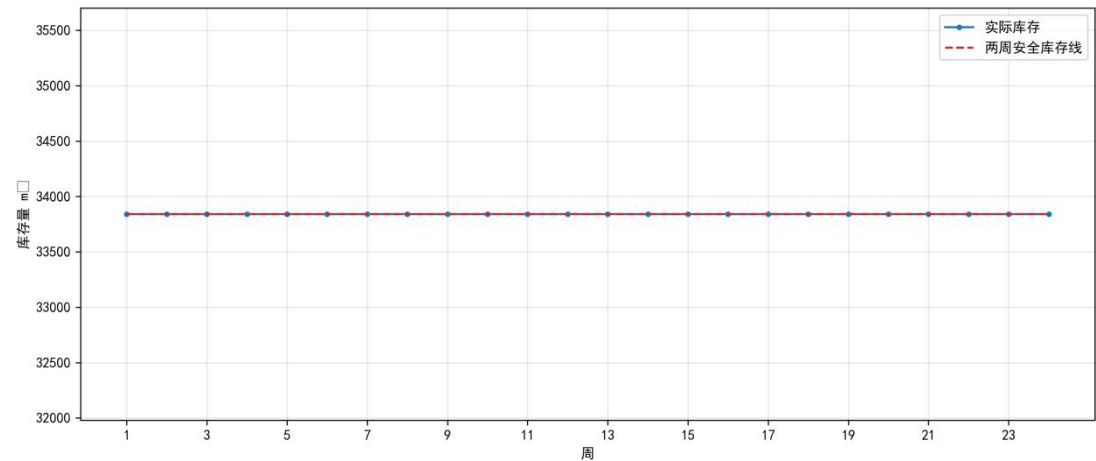


图 9 成本容忍参数 δ 对总成本和 C 类原料占比的影响

——C 类占比已降至 19.7%，而成本仅上浮 5.9%； δ 继续增大到 0.10 时，C 类降为 0 但成本额外上浮 1.9%，边际效益递减。建议选取 $\delta=0.08$ 作为成本与原料结构的平衡点。

（3）问题四产能瓶颈灵敏度。最大可行产能 $P^*=44934.38$ 由三重瓶颈共同决定。若转运商总容量从 48000 立方米/周提升， P^* 可能进一步增大；若供应商履约率整体提升， P^* 也会增大。当前 P^* 对应的每周原材料需求约 29441 立方米，低于转运总容量 48000，说明转运不是最紧瓶颈，供应商供给能力（受履约率限制）是当前主要瓶颈。

6.2 误差分析

（1）数据估计误差。问题一的评价指标、问题二至四的履约率与损耗率均基于 240 周历史数据估计。以履约率为例，采用样本均值作为点估计，存在抽样误差；若历史数据中存在异常周（如供应商临时断供），均值估计可能偏低或偏高。建议对关键参数计算置信区间，或采用稳健估计（如截尾均值）。

（2）模型近似误差。本文将供应商履约的随机性转化为期望履约率，构建确定性 MILP 模型，忽略了供货量的随机波动。实际中若某周供货量低于期望，可能导致库存跌破安全线。可通过蒙特卡洛模拟检验方案在随机供货下的库存达标率，评估方案鲁棒性。

（3）转运损耗估计误差。采用转运商历史平均损耗率作为期望损耗率，忽略了损耗率的周间波动。若实际损耗率高于均值，实际入库量减少，可能影响生

产。问题二的损耗数据显示周间波动较大(第 1 周 937 vs 其他周 34030~53285), 建议核对损耗计算口径, 并对损耗率做波动性分析。

七、模型的评价、改进与推广

7.1 模型优点

(1) 指标体系贴合赛题约束。问题一的四维六指标评价体系将"两周生产库存保障"作为独立维度, 区别于传统供应商评价仅看体量与履约, 紧密扣合赛题安全库存硬性要求。

(2) 客观赋权避免主观偏差。采用熵权法基于数据离散程度自动确定指标权重, 无需专家打分^{[3][5]}, 评价结果可复现、可解释。

(3) 序贯求解降低计算复杂度。问题二将"供应商选择—订购—转运"分解为三个单向依赖阶段, 问题三将"成本基准—结构优化"分解为两阶段 LP, 问题四将"产能评估—方案优化"分解为两阶段, 避免了联合建模的变量爆炸, 各阶段均可精确求解。

(4) 柔性目标处理巧妙。问题三以成本容忍约束替代硬性"多 A 少 C"约束, 既满足题意"尽量"的柔性表述, 又避免了 A 类供给不足导致的模型无解, δ 敏感性分析量化了结构改善的成本代价。

(5) 前后模型自洽。问题二至四的订购模型与转运模型共享同一套约束体系与参数(履约率、损耗率、容量上限), 问题四复用问题二的 MILP 与 GAP 框架, 仅替换产能常数, 保证了模型链路的一致性。

7.2 模型缺点

(1) 确定性近似忽略随机波动。将履约率与损耗率视为确定常数, 未显式建模供货随机性, 方案在极端低供货周可能面临断供风险。

(2) 问题二损耗数据存在异常波动。第 1 周损耗 937 立方米, 其余周在 34030~53285 立方米之间, 波动幅度过大, 可能反映损耗计算公式或数据口径存在问题, 需进一步核查。

(3) 集合覆盖模型使用周均供货量作为能力估计。历史周均供货量是保守估计, 未考虑供应商在激励下可提升供货量的潜力, 可能低估系统供给能力。

(4) 问题四产能评估未考虑设备改造的投资成本。仅从供应链可行性角度评估最大产能, 未权衡产能提升带来的设备改造成本与边际收益, 实际决策中需补充经济性分析。

7.3 改进方向

(1) 引入随机规划或鲁棒优化。将供应商履约率建模为随机变量,采用两阶段随机规划或机会约束规划,在保证一定服务水平(如 95%库存达标率)的前提下优化订购方案,提升方案鲁棒性。

(2) 完善损耗计算与验证。统一损耗率的计算口径(百分比需除以 100),对转运损耗做波动性分析,必要时引入损耗率的随机场景。

(3) 问题四补充产能提升的成本-收益分析。在最大可行产能基础上,引入设备改造成本与产能提升的边际收益,确定经济最优产能(而非仅技术可行产能)。

7.4 模型推广

本文构建的"多准则评价—集合覆盖—多周期 MILP—广义指派"建模框架具有较强的通用性,可推广至以下场景:

(1) 制造业供应链管理。适用于各类制造企业的供应商评价、原材料采购计划与物流配送优化,只需替换原料类别、产能参数与供应商数据即可。

(2) 产能扩张决策。问题四的"瓶颈逐层筛选+二分法搜索最大可行产能"方法可推广至任何面临多重资源约束的产能规划问题,如工厂扩产、仓储中心扩容等。

(3) 多目标柔性决策。问题三的"基准最优+成本容忍约束下结构优化"方法可推广至各类存在"尽量"类柔性目标的决策问题,如能源结构优化、投资组合调整等,通过容忍参数量化目标偏离的代价。

(4) 局限性说明。推广时需注意:本文模型假设需求确定、供应商行为稳定,若应用于需求波动大、供应商进入退出频繁的行业,需增加需求预测与动态供应商管理模块。

八、参考文献

- [1] 运筹学教材编写组. 运筹学[M].4 版.北京:清华大学出版社,2012.
- [2] 司守奎,孙兆亮.数学建模算法与应用[M].2 版.北京:国防工业出版社,2015.
- [3] Chen C H. A Hybrid Multi-Criteria Decision-Making Approach Based on ANP-Entropy TOPSIS for Building Materials Supplier Selection[J]. Entropy, 2021, 23(12):1597.
- [4] Seda M. The Assignment Problem and Its Relation to Logistics Problems[J]. Algorithms, 2022, 15(10):377.
- [5] 葛澜,何东洋.基于熵权 TOPSIS 法供应商评价模型研究[J].建模与仿真,2022,11(05):1223-1230.DOI:10.12677/MOS.2022.115113.

[6] 李垚森,高锦浩,李奕璇,朱俊同,范兴奎.最优供应商选取与原材料订购转运研究[J].合作经济与科技,2024(06):72- 75.

[7] Liu Y, Zhang R Q. Data-driven distributionally robust supplier selection and order allocation problems considering carbon emissions[J]. International Transactions in Operational Research, 2023, 30(4):2145- 2174.

附录 A 支撑材料列表

文件名	类型	简介
<i>clean_data</i>	Python File	数据预处理代码
<i>XLSX</i> 工作表	Python File	熵权-TOPSIS 模型代码
<i>problem2_solve</i>	Python File	问题二模型求解代码
<i>problem3_solve</i>	Python File	两级线性规划+GAP 模型代码
<i>QUICK4</i>	Python File	可行性检验+转运优化模型代码
<i>50SHOP</i>	Python File	问题四取前 50 供应商检验运行代码
<i>show6</i>	Python File	图制作代码
<i>problemshow1</i>	Python File	问题 1 图代码
<i>problemshow2</i>	Python File	问题 2 图代码
<i>preprocessed_data</i>	XLSX 工作表	数据预处理结果
问题一_供应商重要性评价结果	XLSX 工作表	熵权模型评价结果
问题二_中间输出结果	XLSX 工作表	问题二代码运行结果
问题三_输出结果	XLSX 工作表	线性规划和转运优化模型结果
附件 A 订购方案数据结果	XLSX 工作表	订购方案数据结果
附件 B 转运方案数据结果	XLSX 工作表	转运方案数据结果

附录 B 数据预处理代码

```
# -*- coding: utf-8 -*-
"""
data_preprocess.py 预处理脚本
输出 preprocessed_data.xlsx
"""

import numpy as np
import pandas as pd
import warnings
warnings.filterwarnings("ignore")

def main():
    file_in = "F:/8.27 模拟 21 年 c 题/附件 1 近 5 年 402 家供应商的相关数据.xlsx"
```

```

file_out = "preprocessed_data.xlsx"
df_order = pd.read_excel(file_in, sheet_name=0)
df_supply = pd.read_excel(file_in, sheet_name=1)
supplier_id = df_order["供应商 ID"].values
material_type = df_order["材料分类"].values
order_mat = df_order.iloc[:, 2:].values
supply_mat = df_supply.iloc[:, 2:].values
n_supplier = supply_mat.shape[0]
total_week = 240
# 5 代表缺失，替换为 NaN
order_mat = np.where(order_mat == 5, np.nan, order_mat)
supply_mat = np.where(supply_mat == 5, np.nan, supply_mat)
preprocess_list = []
for i in range(n_supplier):
    supply_row = supply_mat[i, :]
    order_row = order_mat[i, :]
    supply_row = np.nan_to_num(supply_row, nan=0.0, posinf=0.0,
neginf=0.0)
    order_row = np.nan_to_num(order_row, nan=0.0, posinf=0.0,
neginf=0.0)
    valid_mask = supply_row > 0
    valid_supply = supply_row[valid_mask]
    valid_week_num = len(valid_supply)
    total_supply = np.sum(supply_row)
    avg_week_supply = total_supply / total_week
    max_week_supply = np.max(supply_row) if valid_week_num>0 else
0.0
    min_week_supply = np.min(valid_supply) if valid_week_num>0 else
0.0
    valid_mean = np.mean(valid_supply) if valid_week_num>0 else 0.0
    valid_std = np.std(valid_supply) if valid_week_num>1 else 0.0
    order_mask = order_row > 1e-6
    has_order = np.any(order_mask)
    order_week_cnt = np.sum(order_mask)
    total_order = np.sum(order_row)
    preprocess_list.append({
        "供应商 ID": supplier_id[i],
        "材料分类": material_type[i],

```

```

        "有效供货周数": valid_week_num,
        "全周期总供货量": total_supply,
        "全周期周均供货量": avg_week_supply,
        "最大单周供货量": max_week_supply,
        "最小单周供货量": min_week_supply,
        "有效周均值": valid_mean,
        "有效周标准差": valid_std,
        "是否下过订单": has_order,
        "订货周数": order_week_cnt,
        "全周期总订货量": total_order
    })

df_pre = pd.DataFrame(preprocess_list)
print(f'总供应商 {len(df_pre)}')
print(f'没有任何订货周的供应商: {np.sum(df_pre['是否下过订单']
]==False)}')
    with pd.ExcelWriter(file_out) as writer:
        df_pre.to_excel(writer, sheet_name="基础统计", index=False)
        pd.DataFrame(order_mat).to_excel(writer, sheet_name="处理后订货
矩阵", index=False, header=False)
        pd.DataFrame(supply_mat).to_excel(writer, sheet_name="处理后供货
矩阵", index=False, header=False)
        print("预处理完成, 输出 preprocessed_data.xlsx")
        print(df_pre.head().round(4))
if __name__ == "__main__":
    main()

```

附录 C 熵权—TOPSIS 综合评价代码

```

# -*- coding: utf-8 -*-
"""

```

```

problem1_solve.py

```

四大维度六大指标:

1. 供货能力维度: 日均供货量、供货贡献占比
2. 履约能力维度: 平均订货履约率
3. 供货连续性维度: 改良条件变异系数、供货频次
4. 两周生产库存保障能力维度: 供货下限保障系数

```

"""
import numpy as np
import pandas as pd
import warnings
warnings.filterwarnings("ignore")
def main():
    pre_file = "F:/8.27 模拟 21 年 c 题/模拟 21c 题代码/preprocessed_data.xlsx"
    out_file = "问题一_供应商重要性评价结果.xlsx"
    total_week = 240 # 论文固定总周数
    # 读取预处理结果
    df_pre = pd.read_excel(pre_file, sheet_name="基础统计")
    order_mat = pd.read_excel(pre_file, sheet_name="处理后订货矩阵",
header=None).values
    supply_mat = pd.read_excel(pre_file, sheet_name="处理后供货矩阵",
header=None).values
    n_supplier = len(df_pre)
    # 全量供应商总供货量（用于供货贡献占比计算）
    all_total_supply = np.sum(df_pre["全周期总供货量"].values)
    index_list = []
    for i in range(n_supplier):
        row = df_pre.iloc[i]
        valid_week_num = row["有效供货周数"]
        total_supply = row["全周期总供货量"]
        avg_week_supply = row["全周期周均供货量"]
        min_week_supply = row["最小单周供货量"]
        valid_mean = row["有效周均值"]
        valid_std = row["有效周标准差"]
        order_row = order_mat[i, :]
        supply_row = supply_mat[i, :]
        # ----- 1. 供货能力维度 -----
        # 指标 1: 日均供货量（论文原文名称）
        avg_day_supply = avg_week_supply / 7.0
        # 指标 2: 供货贡献占比（论文原文名称）
        supply_contribution = total_supply / all_total_supply if
all_total_supply > 1e-6 else 0.0
        # ----- 2. 履约能力维度 -----
        # 指标 3: 平均订货履约率
        fulfill_rate_list = []

```

NaN

```
for j in range(total_week):
    order_ij = order_row[j]
    supply_ij = supply_row[j]
    if order_ij > 1e-6: # 仅对订货量>0 的有效周计算
        fulfill_rate = supply_ij / order_ij
        fulfill_rate_list.append(fulfill_rate)
# 无有效订货周的供应商，用全量有订货供应商的均值填充，避免
NaN

if len(fulfill_rate_list) > 0:
    avg_fulfill_rate = np.mean(fulfill_rate_list)
else:
    avg_fulfill_rate = np.nan # 后续统一用全局均值填充
# ----- 3. 供货连续性维度 -----
# 指标 4: 改良条件变异系数
if valid_week_num > 1 and valid_mean > 1e-6:
    modified_cv = valid_std / valid_mean
else:
    modified_cv = 1.0 # 无有效供货/无波动，赋值为最差值
# 指标 5: 供货频次
supply_frequency = valid_week_num / total_week
# ----- 4. 两周生产库存保障能力维度 -----
# 指标 6: 供货下限保障系数
if avg_week_supply > 1e-6:
    supply_floor_guarantee = min_week_supply / avg_week_supply
else:
    supply_floor_guarantee = 0.0
index_list.append([
    avg_day_supply,
    supply_contribution,
    avg_fulfill_rate,
    modified_cv,
    supply_frequency,
    supply_floor_guarantee
])
# 指标名称 100%匹配论文，一字不改，顺序完全对应
index_cols = [
    "日均供货量",
    "供货贡献占比",
```

```

        "平均订货履约率",
        "改良条件变异系数",
        "供货频次",
        "供货下限保障系数"
    ]

```

```

# 构建指标 DataFrame
df_index = pd.DataFrame(index_list, columns=index_cols)
df_index["供应商 ID"] = df_pre["供应商 ID"]
df_index["材料分类"] = df_pre["材料分类"]
# 平均订货履约率 NaN 填充：用全量有效样本均值填充，避免熵权法
报错

```

```

valid_fulfill = df_index["平均订货履约率"].dropna()
global_mean_fulfill = np.mean(valid_fulfill)
df_index["平均订货履约率"] = df_index["平均订货履约率"]
    ].fillna(global_mean_fulfill)
# 打印指标描述统计，方便论文分析
print("\n==== 六大指标描述统计 =====")
print(df_index[index_cols].describe().round(4))
# ----- 指标正向化与 Min-Max 归一化 -----
def minmax_normalize(df, neg_cols):
    df_norm = df.copy()
    for col in df.columns:
        arr = df_norm[col].values
        minv = np.nanmin(arr)
        maxv = np.nanmax(arr)
        # 避免除零
        if abs(maxv - minv) < 1e-8:
            df_norm[col] = np.full_like(arr, 0.0)
            continue
        # 负向指标正向化
        if col in neg_cols:
            df_norm[col] = (maxv - arr) / (maxv - minv)
        # 正向指标标准化
        else:
            df_norm[col] = (arr - minv) / (maxv - minv)
    return df_norm
# 负向指标：仅改良条件变异系数

```

```

neg_cols = ["改良条件变异系数"]
eval_df = df_index[index_cols].copy()
df_norm = minmax_normalize(eval_df, neg_cols)
# ----- 熵权法计算客观权重 -----
def entropy_weight(norm_df):
    mat = norm_df.values
    m, n = mat.shape
    # 计算概率矩阵
    sum_col = np.sum(mat, axis=0, keepdims=True) + 1e-12
    p = mat / sum_col
    # 增加极小值避免 log(0)产生 NaN
    eps = 1e-9
    p = p + eps
    # 计算信息熵
    e = -1 / np.log(m) * np.sum(p * np.log(p), axis=0)
    # 计算差异系数与权重
    d = 1 - e
    w = d / (np.sum(d) + 1e-12)
    return e, d, w
ent, d_coef, weight = entropy_weight(df_norm)
df_weight = pd.DataFrame({
    "指标": index_cols,
    "熵值 E": ent,
    "差异系数 D": d_coef,
    "权重 W": weight
})
# 按权重降序排序
df_weight = df_weight.sort_values("权重 W",
ascending=False).reset_index(drop=True)
# ----- TOPSIS 法计算综合贴近度 -----
def topsis(norm_df, w):
    mat = norm_df.values
    # 向量规范化
    Z = mat / np.sqrt(np.sum(mat**2, axis=0, keepdims=True) + 1e-12)
    # 加权规范化矩阵
    Zw = Z * w
    # 正/负理想解
    best = np.max(Zw, axis=0)

```



```

        worst = np.min(Zw, axis=0)
        # 欧式距离
        db = np.sqrt(np.sum((Zw - best)**2, axis=1))
        dw = np.sqrt(np.sum((Zw - worst)**2, axis=1))
        # 综合贴近度 (0-1, 越大越好)
        c = dw / (db + dw + 1e-12)
        return c

closeness = topsis(df_norm, weight)
# ----- 结果汇总与排序 -----
df_res = df_index.copy()
df_res["综合贴近度"] = closeness
# 按综合贴近度降序排序
df_res = df_res.sort_values("综合贴近度",
ascending=False).reset_index(drop=True)
# 生成最终排名
df_res["最终排名"] = np.arange(1, len(df_res) + 1)
# 筛选前 50 家重要供应商
df_top50 = df_res.iloc[:50].copy()
# 打印核心结果
print("=" * 60)
print("熵权法指标权重结果 (降序)")
print(df_weight.round(4))
print("=" * 60)
print("综合评价前 10 家重要供应商预览")
print(df_top50[["最终排名", "供应商 ID", "材料分类", "综合贴近度
"]].head(10).round(4))
# 输出结果到 Excel
with pd.ExcelWriter(out_file) as writer:
    df_weight.to_excel(writer, sheet_name="指标权重", index=False)
    df_res.to_excel(writer, sheet_name="全部供应商排名", index=False)
    df_top50.to_excel(writer, sheet_name="TOP50 重要供应商",
index=False)

print(f"\n 问题一求解完成, 输出文件: {out_file}")
if __name__ == "__main__":
    main()

```

附录 D 问题二求解模型代码

```
# -*- coding: utf-8 -*-
"""
输出：
1. 问题二_中间输出结果.xlsx（集合覆盖、订货矩阵、消耗、库存损耗、转运指派原始矩阵）
2. 附件 A：填充【问题 2 的订购方案结果】sheet，无订货留空白
3. 附件 B：填充【问题 2 的转运方案结果】sheet；单元格逗号分隔供应商 ID，无任务为空
"""

import numpy as np
import pandas as pd
import pulp
from openpyxl import load_workbook
import warnings
warnings.filterwarnings("ignore")
# =====全局参数=====
P = 28200
aA, aB, aC = 0.6, 0.66, 0.72
DmaxA = P * aA
DmaxB = P * aB
DmaxC = P * aC
pA, pB, pC = 1.2, 1.1, 1.0
h = 0.01 * pC
I0 = 2 * aC * P
C_trans = 6000
T_total = 24
trans_name_list = ["T001", "T002", "T003", "T004", "T005", "T006", "T007", "T008"]
# 本地路径
BASE = r"F:/8.27 模拟 21 年 c 题/"
FILE_Q1_OUT = BASE + "模拟 21c 题代码/问题一_供应商重要性评价结果.xlsx"
FILE_PRE = BASE + "模拟 21c 题代码/preprocessed_data.xlsx"
FILE_ATT1 = BASE + "附件 1 近 5 年 402 家供应商的相关数据.xlsx"
```

```

FILE_ATT2 = BASE + "附件 2 近 5 年 8 家转运商的相关数据.xlsx"
FILE_ATT_A = BASE + "附件 A 订购方案数据结果.xlsx"
FILE_ATT_B = BASE + "附件 B 转运方案数据结果.xlsx"
FILE_Q2_MID = BASE + "模拟 21c 题代码/问题二_中间输出结果.xlsx"
def main():
    # =====1 读取基础数据=====
    df_q1 = pd.read_excel(FILE_Q1_OUT, sheet_name="TOP50 重要供应商")
    top50_id = df_q1["供应商 ID"].tolist()
    df_pre = pd.read_excel(FILE_PRE, sheet_name="基础统计")
    df_pre = df_pre[df_pre["供应商 ID"].isin(top50_id)].copy().reset_index(drop=True)
    df_pre["local_idx"] = np.arange(len(df_pre))
    Si = df_pre["全周期周均供货量"].values
    mtype_list = df_pre["材料分类"].tolist()
    idxA = [i for i, t in enumerate(mtype_list) if t == "A"]
    idxB = [i for i, t in enumerate(mtype_list) if t == "B"]
    idxC = [i for i, t in enumerate(mtype_list) if t == "C"]
    # 读取附件 1 原始数据
    df_order_raw = pd.read_excel(FILE_ATT1, sheet_name=0)
    df_supply_raw = pd.read_excel(FILE_ATT1, sheet_name=1)
    order_mat = df_order_raw.iloc[:, 2:].values
    supply_mat = df_supply_raw.iloc[:, 2:].values
    order_mat = np.where(order_mat == 5, 0, order_mat)
    supply_mat = np.where(supply_mat == 5, 0, supply_mat)
    mu_list = []
    smax_list = []
    for sid in df_pre["供应商 ID"]:
        pos = list(df_order_raw["供应商 ID"]).index(sid)
        ord_r = order_mat[pos, :]
        sup_r = supply_mat[pos, :]
        mask = ord_r > 1e-6
        ratio = sup_r[mask] / ord_r[mask] if np.sum(mask) > 0 else
np.array([])
        mu = np.mean(ratio) if len(ratio) > 0 else np.nan
        smax = np.max(sup_r)
        mu_list.append(mu)
        smax_list.append(smax)
    mu_valid = [x for x in mu_list if not np.isnan(x)]

```

```

mu_fill = np.mean(mu_valid)
mu = np.array([x if not np.isnan(x) else mu_fill for x in mu_list])
Smax = np.array(smax_list)
# 读取附件 2 转运商, 计算每家转运商平均损耗率 lj
df_trans = pd.read_excel(FILE_ATT2)
trans_mat = df_trans.iloc[:, 1:].values
trans_mat = np.where(trans_mat == 5, np.nan, trans_mat)
lj = np.array([np.nanmean(row) for row in trans_mat])
l_avg_all = np.nanmean(lj)
# =====阶段一: 集合覆盖选供应商=====
def solve_set_cover(lam, use_abc=True):
    n = len(df_pre)
    prob = pulp.LpProblem("supplier_select", pulp.LpMinimize)
    x = [pulp.LpVariable(f"x_{i}", cat="Binary") for i in range(n)]
    prob += pulp.lpSum(x)
    prob += pulp.lpSum([Si[i] * x[i] for i in range(n)]) >= DmaxC
    if use_abc:
        prob += pulp.lpSum([Si[i] * x[i] for i in idxA]) >= lam * DmaxA
        prob += pulp.lpSum([Si[i] * x[i] for i in idxB]) >= lam * DmaxB
        prob += pulp.lpSum([Si[i] * x[i] for i in idxC]) >= lam * DmaxC
    prob.solve(pulp.PULP_CBC_CMD(msg=False))
    selected = [i for i in range(n) if pulp.value(x[i]) > 0.5]
    sumA = sum(Si[i] for i in selected if mtype_list[i] == "A")
    sumB = sum(Si[i] for i in selected if mtype_list[i] == "B")
    sumC = sum(Si[i] for i in selected if mtype_list[i] == "C")
    return {"selected": selected, "sumA": sumA, "sumB": sumB, "sumC":
sumC, "status": pulp.LpStatus[prob.status]}
print("=====阶段一集合覆盖=====")
res0 = solve_set_cover(0.0, use_abc=False)
res02 = solve_set_cover(0.2)
res04 = solve_set_cover(0.4)
res06 = solve_set_cover(0.6)
df_stage1 = pd.DataFrame([
    {"组别": "对照组  $\lambda = 0$ ", "N 供应商": len(res0["selected"]), "sumA":
res0["sumA"], "sumB": res0["sumB"], "sumC": res0["sumC"]},
    {"组别": " $\lambda = 0.2$ ", "N 供应商": len(res02["selected"]), "sumA":
res02["sumA"], "sumB": res02["sumB"], "sumC": res02["sumC"]},
    {"组别": " $\lambda = 0.4$ ", "N 供应商": len(res04["selected"]), "sumA":

```

```

res04["sumA"], "sumB": res04["sumB"], "sumC": res04["sumC"]},
    {"组别": "λ =0.6", "N 供应商": len(res06["selected"]), "sumA":
res06["sumA"], "sumB": res06["sumB"], "sumC": res06["sumC"]},
    ])
print(df_stage1.round(2))
sel = res04["selected"]
df_sel = df_pre.iloc[sel].reset_index(drop=True)
n_sel = len(df_sel)
sel_sid = df_sel["供应商 ID"].tolist()
mu_sel = np.array([mu[i] for i in sel])
smax_sel = np.array([Smax[i] for i in sel])
p_sel = []
for t in df_sel["材料分类"]:
    if t == "A":
        p_sel.append(pA)
    elif t == "B":
        p_sel.append(pB)
    else:
        p_sel.append(pC)
p_sel = np.array(p_sel)
# =====阶段二： 24 周 MIP 订货优化=====
print("\n=====阶段二 24 周订货量求解=====")
prob_milp = pulp.LpProblem("order_plan", pulp.LpMinimize)
D = [[pulp.LpVariable(f"D_{i}_{t}", lowBound=0) for t in range(T_total)]
for i in range(n_sel)]
q = [[pulp.LpVariable(f"q_{k}_{t}", lowBound=0) for t in range(T_total)]
for k in range(3)]
I = [pulp.LpVariable(f"I_{t}", lowBound=0) for t in range(T_total)]

obj = []
for t in range(T_total):
    for i in range(n_sel):
        obj.append(p_sel[i] * mu_sel[i] * D[i][t])
    obj.append(h * I[t])
prob_milp += pulp.lpSum(obj)
for t in range(T_total):
    prob_milp += q[0][t]/aA + q[1][t]/aB + q[2][t]/aC == P
    sum_in = pulp.lpSum([mu_sel[i] * D[i][t] for i in range(n_sel)])

```

```

sumq = q[0][t] + q[1][t] + q[2][t]
if t == 0:
    prob_milp += I[t] == I0 + (1 - l_avg_all)*sum_in - sumq
else:
    prob_milp += I[t] == I[t-1] + (1 - l_avg_all)*sum_in - sumq
prob_milp += I[t] >= 2 * sumq
for i in range(n_sel):
    prob_milp += D[i][t] <= smax_sel[i]
prob_milp += I[-1] >= 2 * (q[0][-1]+q[1][-1]+q[2][-1])
prob_milp.solve(pulp.PULP_CBC_CMD(msg=False))
print("模型求解状态: ", pulp.LpStatus[prob_milp.status])
D_val = np.zeros((n_sel, T_total))
q_val = np.zeros((3, T_total))
I_val = np.zeros(T_total)
for i in range(n_sel):
    for t in range(T_total):
        D_val[i, t] = pulp.value(D[i][t])
for k in range(3):
    for t in range(T_total):
        q_val[k, t] = pulp.value(q[k][t])
for t in range(T_total):
    I_val[t] = pulp.value(I[t])
V_val = mu_sel.reshape(-1,1) * D_val
# =====阶段三 GAP 广义指派: 每周独立求解转运 0-1 规划=====
print("\n=====阶段三转运商 GAP 指派=====")
y_all = []
weekly_loss = []
# trans_assign[t][j]: 第 t 周, 第 j 号转运商, 承接供应商 ID 列表
trans_assign = [ [] for _ in range(8)] for _ in range(T_total) ]
for t in range(T_total):
    vt = V_val[:, t]
    valid_idx = [ii for ii in range(n_sel) if vt[ii] > 1e-6]
    nv = len(valid_idx)
    if nv == 0:
        y_all.append(np.zeros((n_sel,8), dtype=int))
        weekly_loss.append(0.0)
        continue
    p_gap = pulp.LpProblem(f"trans {t}", pulp.LpMinimize)

```

```

        y = [[pulp.LpVariable(f'y_{vi}_{j}', cat="Binary") for j in range(8)]
for vi in range(nv)]
    obj_t = []
    for vi in range(nv):
        real_i = valid_idx[vi]
        for j in range(8):
            obj_t.append(vt[real_i] * lj[j] * y[vi][j])
    p_gap += pulp.lpSum(obj_t)
    # 约束 1: 一个供应商必须指派恰好一个转运商
    for vi in range(nv):
        p_gap += pulp.lpSum(y[vi][j] for j in range(8)) == 1
    # 约束 2: 转运商每周容量上限
    for j in range(8):
        p_gap += pulp.lpSum([ vt[valid_idx[vi]] * y[vi][j] for vi in
range(nv) ]) <= C_trans
    p_gap.solve(pulp.PULP_CBC_CMD(msg=False))
    ymat = np.zeros((n_sel,8), dtype=int)
    for vi in range(nv):
        ri = valid_idx[vi]
        for j in range(8):
            if pulp.value(y[vi][j])>0.5:
                ymat[ri,j]=1
                trans_assign[t][j].append( sel_sid[ri] )
    y_all.append(ymat)

    loss_t = 0.0
    for vi in range(nv):
        ri = valid_idx[vi]
        for j in range(8):
            if ymat[ri,j]==1:
                loss_t += vt[ri] * lj[j]
    weekly_loss.append(loss_t)
total_loss = sum(weekly_loss)

# =====输出评价指标=====
print("\n=====结果指标=====")
total_buy = 0.0
for t in range(T_total):

```

```

        for i in range(n_sel):
            total_buy += p_sel[i] * mu_sel[i] * D_val[i,t]
total_store = h * np.sum(I_val)
total_cost = total_buy + total_store
unit_cost = total_cost/(24*P)
print(f"总采购成本:{total_buy:.2f} 仓储成本:{total_store:.2f} 合
计:{total_cost:.2f}")
print(f"单位产品原材料成本: {unit_cost:.4f}")
safe_cnt = 0
for t in range(T_total):
    sq = q_val[0,t]+q_val[1,t]+q_val[2,t]
    if I_val[t] >= 2*sq -1e-6:
        safe_cnt += 1
safe_rate = safe_cnt / T_total
print(f"库存安全周数: {safe_cnt}/{T_total} 安全率{safe_rate:.2%}")
print(f"总转运损耗: {total_loss:.2f}")

# =====写入附件 A: 问题 2 的订购方案结果=====
print("\n====正在写入附件 A->问题 2 的订购方案结果====")
wbA = load_workbook(FILE_ATT_A)
sheetA_name = "问题 2 的订购方案结果"
wsA = wbA[sheetA_name]

sid_dict = {}
for idx_local in range(n_sel):
    sid_dict[str(sel_sid[idx_local]).strip()] = D_val[idx_local, :]
for row in range(7, wsA.max_row + 1):
    raw_sid = wsA.cell(row=row, column=1).value
    if raw_sid is None:
        continue
    this_id = str(raw_sid).strip()
    for t in range(T_total):
        col = 2 + t
        cell = wsA.cell(row=row, column=col)
        if this_id in sid_dict:
            num = sid_dict[this_id][t]
            if num > 1e-6:
                cell.value = float(num)

```



```

        else:
            cell.value = None
    else:
        cell.value = None
wbA.save(FILE_ATT_A)
print("附件 A 写入完成：无订货单元格全部空白")
# =====写入附件 B：问题 2 的转运方案结果=====
print("\n====正在写入附件 B->问题 2 的转运方案结果====")
wbB = load_workbook(FILE_ATT_B)
sheetB_name = "问题 2 的转运方案结果"
wsB = wbB[sheetB_name]
# 第 7 行开始：T001 在 7 行，T002 在 8 行……T008 在 14 行
for j_idx, tr_name in enumerate(trans_name_list):
    row = 7 + j_idx
    wsB.cell(row=row, column=1).value = tr_name
    for t_idx in range(T_total):
        col = 2 + t_idx
        cell = wsB.cell(row=row, column=col)
        sid_list = trans_assign[t_idx][j_idx]
        if len(sid_list) > 0:
            cell.value = ",".join(sid_list)
        else:
            cell.value = None
wbB.save(FILE_ATT_B)
print("附件 B 写入完成：单元格为逗号分隔供应商 ID，无转运任务单元
格留空")
# =====输出中间结果表格=====
with pd.ExcelWriter(FILE_Q2_MID) as writer:
    df_stage1.to_excel(writer, sheet_name="集合覆盖筛选", index=False)
    df_outD = pd.DataFrame(D_val.T, columns=sel_sid)
    df_outD.insert(0, "周号", np.arange(1, T_total+1))
    df_outD.to_excel(writer, sheet_name="各供应商每周订货 D",
index=False)
    dfq = pd.DataFrame(q_val.T, columns=["A 原料", "B 原料", "C 原料"])
    dfq.insert(0, "周号", np.arange(1, T_total+1))
    dfq.to_excel(writer, sheet_name="每周原料消耗 q", index=False)
    dfi = pd.DataFrame({"周号": np.arange(1, T_total+1), "库存 I": I_val, "每
周损耗": weekly_loss})

```

```

        dfi.to_excel(writer, sheet_name="库存与损耗", index=False)
        # 输出转运指派原始矩阵
        pd.DataFrame(np.array(y_all).reshape(-1,8)).to_excel(writer,
sheet_name="转运 01 矩阵", index=False)
        print("中间文件： 问题二_中间输出结果.xlsx 已生成")
        print("选中供应商列表： ", sel_sid)
    if __name__ == "__main__":
        main()

```

附录 E 两极线性规划+GAP 模型代码

```

# -*- coding:utf-8 -*-
"""
序贯两阶段 LP + GAP 广义指派转运
输出：
1.问题三_输出结果.xlsx： 敏感性分析、 订货矩阵、 原料消耗、 库存、 每周
损耗、 转运指派
2.附件 A： 填充【问题 3 的订购方案结果】sheet， 无订货留空白不写 0
3.附件 B： 填充【问题 3 的转运方案结果】sheet， 逗号分隔供应商 ID， 无任
务为空
"""

import pandas as pd
import pulp
import numpy as np
from openpyxl import load_workbook
import warnings
warnings.filterwarnings("ignore")

# =====全局参数&本地路径=====
P = 28200
aA, aB, aC = 0.6, 0.66, 0.72
pA, pB, pC = 1.2, 1.1, 1.0
h = 0.01 * pC
ct = 0.005 * pC
I0 = 2 * aC * P
C_trans_max = 6000
T_total = 24

```

```

trans_name_list = ["T001", "T002", "T003", "T004", "T005", "T006", "T007",
"T008"]
delta_candidates = [0.02,0.05,0.08,0.10]    # 成本容忍度做敏感性分析
delta_choose = 0.08                        # 最终选用的  $\delta$ 
BASE_PATH = r"F:/8.27 模拟 21 年 c 题/"
Q2_OUTPUT_FILE = BASE_PATH + "模拟 21c 题代码/问题二_中间输出结果.xlsx"
PREPROCESSED_FILE = BASE_PATH + " 模 拟 21c 题 代 码 /preprocessed_data.xlsx"
FILE_ATT2 = BASE_PATH + "附件 2 近 5 年 8 家转运商的相关数据.xlsx"
ATTACHMENT_A_FILE = BASE_PATH + "附件 A 订购方案数据结果.xlsx"
ATTACHMENT_B_FILE = BASE_PATH + "附件 B 转运方案数据结果.xlsx"
Q3_OUTPUT_FILE = BASE_PATH + "模拟 21c 题代码/问题三_输出结果.xlsx"
SHEET_A_TARGET = "问题 3 的订购方案结果"
SHEET_B_TARGET = "问题 3 的转运方案结果"
ORDER_THRESHOLD = 1.0
def main():
    # =====1 读取数据：问题二输出选中供应商集合 =====
    print("====步骤 1 读取基础数据====")
    df_D2 = pd.read_excel(Q2_OUTPUT_FILE, sheet_name="各供应商每周订货 D")
    sel_sid = df_D2.columns.tolist()[1:]
    print(f"问题二选出供应商数量:{len(sel_sid)}")
    df_pre = pd.read_excel(PREPROCESSED_FILE)
    sel_supplier_df = df_pre[df_pre[" 供 应 商 ID"].isin(sel_sid)].copy().reset_index(drop=True)
    n_sel = len(sel_supplier_df)
    supplier_id_list = sel_supplier_df["供应商 ID"].tolist()
    mu_arr = sel_supplier_df["有效周均值"].values
    Smax_arr = sel_supplier_df["最大单周供货量"].values
    mtype_list = sel_supplier_df["材料分类"].tolist()
    sup_dict = {}
    idxA_list, idxC_list = [], []
    for i, sid in enumerate(supplier_id_list):
        sup_dict[sid] = {
            "mu": mu_arr[i],
            "Smax": Smax_arr[i],

```

```

        "type": mtype_list[i]
    }
    if mtype_list[i] == "A":
        idxA_list.append(i)
    if mtype_list[i] == "C":
        idxC_list.append(i)

weeks = list(range(1, T_total+1))
# 读取附件 2, 得到每家转运商平均损耗率 lj, 全局平均损耗率 l_avg_all
df_trans = pd.read_excel(FILE_ATT2)
trans_mat = df_trans.iloc[:, 1:].values
trans_mat = np.where(trans_mat == 5, np.nan, trans_mat)
lj = np.array([np.nanmean(row) for row in trans_mat])
l_avg_all = np.nanmean(lj)
# =====阶段 1: 基准 LP, 求最小综合成本 Z1_star =====
print("\n====步骤 2 阶段一: 求解基准最优 LP, 获取 Z1*====")
prob_base = pulp.LpProblem("Base_LP", pulp.LpMinimize)
D_base = [[pulp.LpVariable(f'Dbase_{i}_{t}', lowBound=0) for t in weeks]
for i in range(n_sel)]
q_base = [[pulp.LpVariable(f'qbase_{k}_{t}', lowBound=0) for t in weeks]
for k in range(3)]
I_base = [pulp.LpVariable(f'Ibase_{t}', lowBound=0) for t in weeks]
# 【目标函数: 综合成本=采购+仓储+转运+损耗经济损失】
obj_base_expr = []
for t_idx, t in enumerate(weeks):
    sum_rec_exp = pulp.lpSum([mu_arr[i]*D_base[i][t_idx] for i in
range(n_sel)])
    cost_buy = pulp.lpSum([
        (pA if mtype_list[i]=="A" else (pB if mtype_list[i]=="B" else
pC))
        * mu_arr[i] * D_base[i][t_idx] for i in range(n_sel)
    ])
    cost_store = h * I_base[t_idx]
    cost_trans = ct * (1-l_avg_all) * sum_rec_exp
    cost_loss = pulp.lpSum([
        (pA if mtype_list[i]=="A" else (pB if mtype_list[i]=="B" else
pC))
        * mu_arr[i] * l_avg_all * D_base[i][t_idx] for i in range(n_sel)

```

```

    ])
    obj_base_expr.append(cost_buy + cost_store + cost_trans + cost_loss)
prob_base += pulp.lpSum(obj_base_expr)

# 约束集合
for t_idx, t in enumerate(weeks):
    # 满产约束
    prob_base += q_base[0][t_idx]/aA + q_base[1][t_idx]/aB +
q_base[2][t_idx]/aC == P
    sum_rec_exp = pulp.lpSum([mu_arr[i] * D_base[i][t_idx] for i in
range(n_sel)])
    sum_q = q_base[0][t_idx] + q_base[1][t_idx] + q_base[2][t_idx]
    if t_idx == 0:
        prob_base += I_base[t_idx] == I0 + (1-l_avg_all)*sum_rec_exp -
sum_q
    else:
        prob_base += I_base[t_idx] == I_base[t_idx-1] +
(1-l_avg_all)*sum_rec_exp - sum_q
        prob_base += I_base[t_idx] >= 2 * sum_q
        for i in range(n_sel):
            prob_base += D_base[i][t_idx] <= Smax_arr[i]
prob_base += I_base[-1] >= 2*(q_base[0][-1]+q_base[1][-1]+q_base[2][-1])

prob_base.solve(pulp.PULP_CBC_CMD(msg=False))
stat_base = pulp.LpStatus[prob_base.status]
if stat_base != "Optimal":
    raise Exception("基准 LP 不可行，请检查供应商供给能力")
Z1_star = pulp.value(prob_base.objective)
print(f"基准模型状态:{stat_base}, 基准最小综合成本 Z1* =
{Z1_star:.2f}")

# =====阶段 2: 对每个  $\delta$  求解结构优化 LP, 敏感性分析 =====
print("\n====步骤 3 阶段二: 不同  $\delta$  结构优化 LP====")
sens_result = []
sol_dict = dict()
for delta in delta_candidates:
    print(f"\n---求解  $\delta = \{\delta\}$  ---")
    prob_struct = pulp.LpProblem(f"Struct_delta_{delta}",

```

```

pulp.LpMinimize)
    D_var = [[pulp.LpVariable(f"D_{i}_{t}", lowBound=0) for t in weeks]
for i in range(n_sel)]
    q_var = [[pulp.LpVariable(f"q_{k}_{t}", lowBound=0) for t in weeks]
for k in range(3)]
    I_var = [pulp.LpVariable(f"I_{t}", lowBound=0) for t in weeks]

# 目标函数 min( $\sum C$  期望供货 -  $\sum A$  期望供货), 文档 4.2
    sumC_exp = pulp.lpSum([mu_arr[i]*D_var[i][t_idx] for i in idxC_list
for t_idx,_ in enumerate(weeks)])
    sumA_exp = pulp.lpSum([mu_arr[i]*D_var[i][t_idx] for i in idxA_list
for t_idx,_ in enumerate(weeks)])
    prob_struct += (sumC_exp - sumA_exp)
# 构建综合成本表达式, 用于成本容忍约束
    total_cost_expr = []
    for t_idx,t in enumerate(weeks):
        sum_rec_exp = pulp.lpSum([mu_arr[i]*D_var[i][t_idx] for i in
range(n_sel)])
        cost_buy = pulp.lpSum([
            (pA if mtype_list[i]=="A" else (pB if mtype_list[i]=="B"
else pC))
            * mu_arr[i] * D_var[i][t_idx] for i in range(n_sel)
        ])
        cost_store = h * I_var[t_idx]
        cost_trans = ct * (1-l_avg_all) * sum_rec_exp
        cost_loss = pulp.lpSum([
            (pA if mtype_list[i]=="A" else (pB if mtype_list[i]=="B"
else pC))
            * mu_arr[i] * l_avg_all * D_var[i][t_idx] for i in
range(n_sel)
        ])
        total_cost_expr.append(cost_buy + cost_store + cost_trans +
cost_loss)

    Z_total = pulp.lpSum(total_cost_expr)
# 新增成本容忍约束  $Z \leq (1+\delta)*Z1\_star$ 
    prob_struct += Z_total <= (1.0+delta)*Z1_star

```

```

# 复制全部基准模型约束
for t_idx, t in enumerate(weeks):
    prob_struct += q_var[0][t_idx]/aA + q_var[1][t_idx]/aB +
q_var[2][t_idx]/aC == P
    sum_rec_exp = pulp.lpSum([mu_arr[i] * D_var[i][t_idx] for i in
range(n_sel)])
    sum_q = q_var[0][t_idx] + q_var[1][t_idx] + q_var[2][t_idx]
    if t_idx == 0:
        prob_struct += I_var[t_idx] == I0 +
(1-l_avg_all)*sum_rec_exp - sum_q
    else:
        prob_struct += I_var[t_idx] == I_var[t_idx-1] +
(1-l_avg_all)*sum_rec_exp - sum_q
        prob_struct += I_var[t_idx] >= 2 * sum_q
    for i in range(n_sel):
        prob_struct += D_var[i][t_idx] <= Smax_arr[i]
prob_struct += I_var[-1] >=
2*(q_var[0][-1]+q_var[1][-1]+q_var[2][-1])

```

```

prob_struct.solve(pulp.PULP_CBC_CMD(msg=False))
stat_s = pulp.LpStatus[prob_struct.status]
if stat_s != "Optimal":
    print(f'δ={delta} 无可行解，跳过')
    continue
# 提取解
D_sol = np.zeros((n_sel, T_total))
q_sol = np.zeros((3, T_total))
I_sol = np.zeros(T_total)
for i in range(n_sel):
    for t_idx, _ in enumerate(weeks):
        D_sol[i, t_idx] = pulp.value(D_var[i][t_idx])
for k in range(3):
    for t_idx, _ in enumerate(weeks):
        q_sol[k, t_idx] = pulp.value(q_var[k][t_idx])
for t_idx, _ in enumerate(weeks):
    I_sol[t_idx] = pulp.value(I_var[t_idx])
Z_this = pulp.value(Z_total)
V_sol = mu_arr.reshape(-1,1) * D_sol

```

```

sumA_val = np.sum(V_sol[idxA_list,:])
sumC_val = np.sum(V_sol[idxC_list,:])
sumAll_val = np.sum(V_sol)
sens_result.append({
    "delta":delta,
    "status":stat_s,
    "Z_total":Z_this,
    "Z_upper":(1+delta)*Z1_star,
    "sumA_exp":sumA_val,
    "sumC_exp":sumC_val,
    "ratioA": sumA_val/sumAll_val,
    "ratioC": sumC_val/sumAll_val
})
sol_dict[delta] = {"D":D_sol,"q":q_sol,"I":I_sol,"V":V_sol}
print(f"     $\delta$  = {delta}    | 总成本 {Z_this:.2f}    | A 占比
{sumA_val/sumAll_val:.2%} | C 占比 {sumC_val/sumAll_val:.2%}")

```

```

df_sens = pd.DataFrame(sens_result)
print("\n====敏感性分析汇总表====")
print(df_sens.round(4))
# 取选定  $\delta$  的方案
if delta_choose not in sol_dict:
    raise Exception(f'选定 delta={delta_choose} 无可行解,请更换 delta')
sol_final = sol_dict[delta_choose]
D_final = sol_final["D"]
q_final = sol_final["q"]
I_final = sol_final["I"]
V_final = sol_final["V"]
# =====阶段3 GAP 广义指派转运(每周独立 0-1 规划) =====
print("\n====步骤4 GAP 转运指派求解====")
trans_assign = [ [] for _ in range(8)] for _ in range(T_total) ]
weekly_loss = []
for t_idx in range(T_total):
    vt = V_final[:, t_idx]
    valid_idx = [ii for ii in range(n_sel) if vt[ii]>1e-6]
    nv = len(valid_idx)
    if nv == 0:

```



```

        weekly_loss.append(0.0)
        continue
    p_gap = pulp.LpProblem(f'gap_t{t_idx}', pulp.LpMinimize)
    y = [[pulp.LpVariable(f'y_{vi}_{j}',cat="Binary") for j in range(8)]
for vi in range(nv)]
    obj_t = []
    for vi in range(nv):
        real_i = valid_idx[vi]
        for j in range(8):
            obj_t.append(vt[real_i]*lj[j]*y[vi][j])
    p_gap += pulp.lpSum(obj_t)
    #约束 1: 每个供应商恰好一个转运商
    for vi in range(nv):
        p_gap += pulp.lpSum(y[vi][j] for j in range(8)) == 1
    #约束 2: 转运商容量上限
    for j in range(8):
        p_gap += pulp.lpSum([ vt[valid_idx[vi]]*y[vi][j] for vi in
range(nv) ]) <= C_trans_max
    p_gap.solve(pulp.PULP_CBC_CMD(msg=False))
    ymat = np.zeros((n_sel,8),dtype=int)
    for vi in range(nv):
        ri = valid_idx[vi]
        for j in range(8):
            if pulp.value(y[vi][j])>0.5:
                ymat[ri,j]=1
                trans_assign[t_idx][j].append(supplier_id_list[ri])
    #计算本周损耗
    loss_t=0.0
    for vi in range(nv):
        ri = valid_idx[vi]
        for j in range(8):
            if ymat[ri,j]==1:
                loss_t += vt[ri]*lj[j]
    weekly_loss.append(loss_t)
total_loss = sum(weekly_loss)
print(f'24 周总转运损耗: {total_loss:.2f}')
# =====结果整理, 准备输出 Excel 与附件 A/B =====
# 订货字典, 用于附件 A 填充

```

```

order_dict = {}
for i, sid in enumerate(supplier_id_list):
    order_dict[str(sid).strip()] = D_final[i,:].tolist()

df_D_out = pd.DataFrame(D_final.T, columns=supplier_id_list)
df_D_out.insert(0,"周号",np.arange(1,T_total+1))
df_q_out = pd.DataFrame(q_final.T, columns=["A 原料消耗","B 原料消耗",
"C 原料消耗"])
df_q_out.insert(0,"周号",np.arange(1,T_total+1))
df_inv_out = pd.DataFrame({
    "周号":np.arange(1,T_total+1),
    "周末库存 I":I_final,
    "每周转运损耗":weekly_loss
})

with pd.ExcelWriter(Q3_OUTPUT_FILE) as w:
    df_sens.to_excel(w, sheet_name="δ 敏感性分析", index=False)
    df_D_out.to_excel(w, sheet_name="各 供应商 每周 订货 D3",
index=False)
    df_q_out.to_excel(w, sheet_name="每周原料消耗 q3", index=False)
    df_inv_out.to_excel(w, sheet_name="库存与损耗 3", index=False)
print("中间结果已写入： 问题三_输出结果.xlsx")
# =====填充附件 A 订购方案 =====
print("\n====步骤 5 填充附件 A====")
wbA = load_workbook(ATTACHMENT_A_FILE)
wsA = wbA[SHEET_A_TARGET]
hit_a = 0
for row in range(7, wsA.max_row+1):
    raw_sid = wsA.cell(row=row,column=1).value
    if raw_sid is None:
        continue
    sid_cur = str(raw_sid).strip()
    for t_idx in range(T_total):
        col = 2 + t_idx
        cell = wsA.cell(row=row, column=col)
        if sid_cur in order_dict:
            val = order_dict[sid_cur][t_idx]
            cell.value = float(val) if val>ORDER_THRESHOLD else

```

None

```
        else:
            cell.value = None
        if sid_cur in order_dict:
            hit_a += 1
wbA.save(ATTACHMENT_A_FILE)
print(f"附件 A 填充完成，匹配供应商 {hit_a} 家")

# =====填充附件 B 转运方案 =====
print("\n====步骤 6 填充附件 B====")
wbB = load_workbook(ATTACHMENT_B_FILE)
wsB = wbB[SHEET_B_TARGET]
for j_idx, tr_name in enumerate(trans_name_list):
    row = 7 + j_idx
    wsB.cell(row=row, column=1).value = tr_name
    for t_idx in range(T_total):
        col = 2 + t_idx
        cell = wsB.cell(row=row, column=col)
        sid_list = trans_assign[t_idx][j_idx]
        cell.value = ",".join(sid_list) if len(sid_list)>0 else None
wbB.save(ATTACHMENT_B_FILE)
print("附件 B 填充完成，转运方案已写入")
# =====打印实施效果指标 =====
print("\n=====实施效果指标=====")
safe_cnt = 0
for t_idx in range(T_total):
    sq = q_final[0,t_idx]+q_final[1,t_idx]+q_final[2,t_idx]
    if I_final[t_idx] >= 2*sq -1e-6:
        safe_cnt += 1
safe_rate = safe_cnt / T_total
avg_inv = np.mean(I_final)
total_raw_consume = np.sum(q_final)
inv_turnover = total_raw_consume / avg_inv if avg_inv>1e-6 else 0.0
print(f"安全库存达标周数： {safe_cnt}/{T_total}，达标率
{safe_rate:.2%}")
print(f"平均库存： {avg_inv:.2f}，库存周转率 {inv_turnover:.3f}")
print(f"24 周总转运损耗： {total_loss:.2f}")
print("\n全部任务完成：附件 A、附件 B、中间输出文件已生成")
```

```
if __name__ == "__main__":  
    main()
```

附录 F 可行性检验代码

```
from openpyxl import load_workbook  
from openpyxl.utils import get_column_letter  
from copy import copy  
  
# ===== 1. 【配置区】 附件路径 =====  
SUPPLIER_DATA_PATH = "F:/8.27 模拟 21 年 c 题/附件 1 近 5 年 402 家供  
应商的相关数据.xlsx"  
TRANSIT_DATA_PATH = "F:/8.27 模拟 21 年 c 题/附件 2 近 5 年 8 家转运商  
的相关数据.xlsx"  
TEMPLATE_A_PATH = "F:/8.27 模拟 21 年 c 题/附件 A 订购方案数据结  
果.xlsx"  
TEMPLATE_B_PATH = "F:/8.27 模拟 21 年 c 题/附件 B 转运方案数据结  
果.xlsx"  
  
# 题目固定参数  
P0 = 28200  
r_A = 0.6  
r_B = 0.66  
r_C = 0.72  
U = 6000  
C_C = 1.0  
C_A = 1.2 * C_C  
C_B = 1.1 * C_C  
c_tr = 0.05  
c_h = 0.02  
I0 = 80000  
T_24 = list(range(1, 25))  
SEARCH_LOW = P0  
SEARCH_HIGH = 45000  
EPSILON = 100  
MAX_BISECT_STEP = 8  
M_LOCAL = 50000  
SOLVE_TIME_LIMIT = 100
```

```

def get_solver():
    """旧版 pulp 兼容，仅 CBC 求解器，仅保留支持参数"""
    solver = pulp.PULP_CBC_CMD(
        msg=False,
        timeLimit=SOLVE_TIME_LIMIT
    )
    return solver

def load_supplier_data(path):
    df_order = pd.read_excel(path, sheet_name='企业的订货量 (m³) ')
    df_supply = pd.read_excel(path, sheet_name='供应商的供货量 (m³) ')

    df_order = df_order.sort_values("供应商 ID").reset_index(drop=True)
    df_supply = df_supply.sort_values("供应商 ID").reset_index(drop=True)
    supplier_ids = df_order["供应商 ID"].tolist()
    type_map = {"A": 0, "B": 1, "C": 2}
    supplier_type = np.array([type_map[t] for t in df_order["材料分类"]])
    order_cols = [c for c in df_order.columns if c.startswith("W")]
    supply_cols = [c for c in df_supply.columns if c.startswith("W")]
    order_matrix = df_order[order_cols].to_numpy()
    supply_matrix = df_supply[supply_cols].to_numpy()
    mu = np.where(order_matrix == 0, 0, supply_matrix /
np.maximum(order_matrix, 1))
    mu = np.mean(mu, axis=1)
    mu = np.clip(mu, 0.3, 1.0)
    return supplier_ids, supplier_type, mu

def load_transit_data(path):
    df = pd.read_excel(path, sheet_name="运输损耗率 (%) ")
    df = df.sort_values("转运商 ID").reset_index(drop=True)
    transit_ids = df["转运商 ID"].tolist()
    loss_cols = [f"W {t:03d}" for t in range(1, 25)]
    l_jt = df[loss_cols].to_numpy()
    return transit_ids, l_jt

supplier_ids, supplier_type, mu = load_supplier_data(SUPPLIER_DATA_PATH)
transit_ids, l_jt = load_transit_data(TRANSIT_DATA_PATH)
n_supplier = len(supplier_ids)
n_transit = len(transit_ids)

```

```

print(f"读取数据： 供应商 {n_supplier} 家， 转运商 {n_transit} 家")
def calc_D(P_new):
    frac_A = np.mean(supplier_type == 0)
    frac_B = np.mean(supplier_type == 1)
    frac_C = np.mean(supplier_type == 2)
    D = P_new * (frac_A * r_A + frac_B * r_B + frac_C * r_C)
    return D

# =====阶段一： 可行性模型=====
def is_feasible(P_new):
    D = calc_D(P_new)
    I_min = 2 * D
    prob = pulp.LpProblem("FeasibilityCheck", pulp.LpMinimize)
    prob += 0

    q = [[pulp.LpVariable(f'q_{i}_{t}', lowBound=0) for t in T_24] for i in
range(n_supplier)]
    s = [[pulp.LpVariable(f's_{i}_{t}', lowBound=0) for t in T_24] for i in
range(n_supplier)]
    x = [[[pulp.LpVariable(f'x_{i}_{j}_{t}', cat=pulp.LpBinary) for t in T_24]
for j in range(n_transit)] for i in range(n_supplier)]
    z = [[[pulp.LpVariable(f'z_{i}_{j}_{t}', lowBound=0) for t in T_24] for j
in range(n_transit)] for i in range(n_supplier)]
    R = {t: pulp.LpVariable(f'R_{t}', lowBound=0) for t in T_24}
    I = {t: pulp.LpVariable(f'I_{t}', lowBound=0) for t in T_24}

    for i in range(n_supplier):
        for t in T_24:
            prob += s[i][t - 1] == mu[i] * q[i][t - 1]

    # z = s*x 线性化， 删除错误下界约束
    for i in range(n_supplier):
        for j in range(n_transit):
            for t in T_24:
                zi = z[i][j][t - 1]
                si = s[i][t - 1]
                xi = x[i][j][t - 1]
                prob += zi <= M_LOCAL * xi
                prob += zi <= si

```

```

for t in T_24:
    expr = pulp.lpSum([z[i][j][t - 1] * (1.0 - l_jt[j, t - 1] / 100.0)
                        for i in range(n_supplier) for j in
range(n_transit)])
    prob += R[t] == expr

for t in T_24:
    prevI = I0 if t == 1 else I[t - 1]
    prob += I[t] == prevI + R[t] - D

for t in T_24:
    prob += I[t] >= I_min
    prob += I[t] >= 0

for i in range(n_supplier):
    for t in T_24:
        prob += pulp.lpSum([x[i][j][t - 1] for j in range(n_transit)]) <= 1

for j in range(n_transit):
    for t in T_24:
        expr = pulp.lpSum([z[i][j][t - 1] for i in range(n_supplier)])
        prob += expr <= U

solver = get_solver()
prob.solve(solver)
status = pulp.LpStatus[prob.status]
return status == "Optimal"

#二分搜索
low = SEARCH_LOW
high = SEARCH_HIGH
best_Pnew = SEARCH_LOW
step_count = 0
print("\n====阶段一： 二分搜索最大可行产能====")
while high - low > EPSILON and step_count < MAX_BISECT_STEP:
    mid = (low + high) / 2
    flag = is_feasible(mid)
    step_count += 1

```

```

        if flag:
            best_Pnew = mid
            low = mid
            print(f"P_new={mid:.1f} 可行")
        else:
            high = mid
            print(f"P_new={mid:.1f} 不可行")
    P_new = best_Pnew
    D_fixed = calc_D(P_new)
    Imin_fixed = 2 * D_fixed
    print(f"\n>>> 阶段一结果： P_new={P_new:.2f}, D={D_fixed:.2f},
I_min={Imin_fixed:.2f}")

```

附录 G 转运优化模型

```

import pulp
import numpy as np
import pandas as pd
from openpyxl import load_workbook
from openpyxl.utils import range_boundaries
from copy import copy
# ===== 1. 【配置区】附件路径 =====
SUPPLIER_DATA_PATH = "F:/8.27 模拟 21 年 c 题/附件 1 近 5 年 402 家供
应商的相关数据.xlsx"
TRANSIT_DATA_PATH = "F:/8.27 模拟 21 年 c 题/附件 2 近 5 年 8 家转运商
的相关数据.xlsx"
TEMPLATE_A_PATH = "F:/8.27 模拟 21 年 c 题/附件 A 订购方案数据结
果.xlsx"
TEMPLATE_B_PATH = "F:/8.27 模拟 21 年 c 题/附件 B 转运方案数据结
果.xlsx"
SHEET_Q4 = "问题 4 的订购方案结果"
SHEET_Q5 = "问题 4 的转运方案结果"
# 题目固定参数
P0 = 28200
r_A = 0.6
r_B = 0.66

```



```

r_C = 0.72
U = 6000
C_C = 1.0
C_A = 1.2 * C_C
C_B = 1.1 * C_C
c_tr = 0.05
c_h = 0.02
I0 = 80000
T_24 = list(range(1, 25))
M_LOCAL = 50000
SOLVE_TIME_LIMIT = 300
TEST_SUPPLIER_NUM = 50    # 只取前 50 家
def get_solver():
    """旧版 pulp 兼容，CBC 求解器"""
    solver = pulp.PULP_CBC_CMD(
        msg=False,
        timeLimit=SOLVE_TIME_LIMIT
    )
    return solver
def load_supplier_data(path, take_n=None):
    df_order = pd.read_excel(path, sheet_name='企业的订货量 (m³) ')
    df_supply = pd.read_excel(path, sheet_name='供应商的供货量 (m³) ')

    df_order = df_order.sort_values("供应商 ID").reset_index(drop=True)
    df_supply = df_supply.sort_values("供应商 ID").reset_index(drop=True)
    if take_n is not None:
        df_order = df_order.head(take_n).copy()
        df_supply = df_supply.head(take_n).copy()
    supplier_ids = df_order["供应商 ID"].tolist()
    type_map = {"A": 0, "B": 1, "C": 2}
    supplier_type = np.array([type_map[t] for t in df_order["材料分类"]])
    order_cols = [c for c in df_order.columns if c.startswith("W")]
    supply_cols = [c for c in df_supply.columns if c.startswith("W")]
    order_matrix = df_order[order_cols].to_numpy()
    supply_matrix = df_supply[supply_cols].to_numpy()
    mu = np.where(order_matrix == 0, 0, supply_matrix /
np.maximum(order_matrix, 1))
    mu = np.mean(mu, axis=1)

```

```

mu = np.clip(mu, 0.3, 1.0)
return supplier_ids, supplier_type, mu
def load_transit_data(path):
    df = pd.read_excel(path, sheet_name="运输损耗率 (%) ")
    df = df.sort_values("转运商 ID").reset_index(drop=True)
    transit_ids = df["转运商 ID"].tolist()
    loss_cols = [f"W{t:03d}" for t in range(1, 25)]
    l_jt = df[loss_cols].to_numpy()
    return transit_ids, l_jt
# 读取前 50 家供应商
supplier_ids, supplier_type, mu = load_supplier_data(SUPPLIER_DATA_PATH,
take_n=TEST_SUPPLIER_NUM)
transit_ids, l_jt = load_transit_data(TRANSIT_DATA_PATH)
n_supplier = len(supplier_ids)
n_transit = len(transit_ids)
print(f'读取数据： 供应商 {n_supplier} 家， 转运商 {n_transit} 家')
def calc_D(P_new):
    frac_A = np.mean(supplier_type == 0)
    frac_B = np.mean(supplier_type == 1)
    frac_C = np.mean(supplier_type == 2)
    D = P_new * (frac_A * r_A + frac_B * r_B + frac_C * r_C)
    return D
# 手动指定产能： 50 家样本占比改变， 有可能无解
P_new = 44934.38
D_fixed = calc_D(P_new)
Imin_fixed = 2 * D_fixed
print(f'\n>>> 设 置      :      P_new={P_new:.2f},      D={D_fixed:.2f},
I_min={Imin_fixed:.2f}')

```

```

# =====阶段二： 优化模型=====
def solve_order_transit(P_fixed):
    D = calc_D(P_fixed)
    Imin = 2 * D
    prob = pulp.LpProblem("OrderTransPlan", pulp.LpMinimize)

    q = [[pulp.LpVariable(f'q_{i}_{t}', lowBound=0) for t in T_24] for i in
range(n_supplier)]

```

```

s = [[pulp.LpVariable(f"s_{i}_{t}", lowBound=0) for t in T_24] for i in
range(n_supplier)]
x = [[[pulp.LpVariable(f"x_{i}_{j}_{t}", cat=pulp.LpBinary) for t in T_24]
for j in range(n_transit)] for i in range(n_supplier)]
z = [[[pulp.LpVariable(f"z_{i}_{j}_{t}", lowBound=0) for t in T_24] for j
in range(n_transit)] for i in range(n_supplier)]
R = {t: pulp.LpVariable(f"R_{t}", lowBound=0) for t in T_24}
I = {t: pulp.LpVariable(f"I_{t}", lowBound=0) for t in T_24}

cost_list = []
for t in T_24:
    cp_expr = pulp.lpSum([
        (C_A * q[i][t-1] if supplier_type[i]==0 else
         C_B * q[i][t-1] if supplier_type[i]==1 else C_C * q[i][t-1])
        for i in range(n_supplier)
    ])
    ctr_expr = c_tr * pulp.lpSum([s[i][t-1] for i in range(n_supplier)])
    ch_expr = c_h * I[t]
    cost_list.append(cp_expr + ctr_expr + ch_expr)
total_cost = pulp.lpSum(cost_list)
prob += total_cost
for i in range(n_supplier):
    for t in T_24:
        prob += s[i][t - 1] == mu[i] * q[i][t - 1]

# z = s*x 线性化
for i in range(n_supplier):
    for j in range(n_transit):
        for t in T_24:
            zi = z[i][j][t - 1]
            si = s[i][t - 1]
            xi = x[i][j][t - 1]
            prob += zi <= M_LOCAL * xi
            prob += zi <= si
for t in T_24:
    expr = pulp.lpSum([z[i][j][t - 1] * (1.0 - l_jt[j, t - 1] / 100.0)
                        for i in range(n_supplier) for j in
range(n_transit)])

```

```

        prob += R[t] == expr
    for t in T_24:
        prevI = I0 if t == 1 else I[t - 1]
        prob += I[t] == prevI + R[t] - D

    for t in T_24:
        prob += I[t] >= Imin
        prob += I[t] >= 0
    for i in range(n_supplier):
        for t in T_24:
            prob += pulp.lpSum([x[i][j][t - 1] for j in range(n_transit)]) <= 1
    for j in range(n_transit):
        for t in T_24:
            expr = pulp.lpSum([z[i][j][t - 1] for i in range(n_supplier)])
            prob += expr <= U
    solver = get_solver()
    prob.solve(solver)
    status = pulp.LpStatus[prob.status]
    print(f'阶段二求解状态: {status}')

```

```

q_res = np.zeros((n_supplier, 24))
s_res = np.zeros((n_supplier, 24))
x_res = np.zeros((n_supplier, n_transit, 24), dtype=int)
I_res = np.zeros(24)
for i in range(n_supplier):
    for tidx, _ in enumerate(T_24):
        q_res[i, tidx] = pulp.value(q[i][tidx]) or 0
        s_res[i, tidx] = pulp.value(s[i][tidx]) or 0
        for j in range(n_transit):
            val = pulp.value(x[i][j][tidx])
            x_res[i, j, tidx] = int(val) if val is not None else 0
for tidx, t in enumerate(T_24):
    I_res[tidx] = pulp.value(I[t]) or 0
totalZ = pulp.value(total_cost) or 0
return q_res, s_res, x_res, I_res, totalZ

```

```

q_matrix, s_matrix, x_matrix, I_arr, totalZ = solve_order_transit(P_new)

```

```

print(f"\n24 周总费用 {totalZ:.2f}")
print(f"各周库存 {np.round(I_arr, 2)}")
# =====填充附件 A=====
def fill_template_A(template_path, sheet_name, supplier_ids, q_matrix):
    wb = load_workbook(template_path)
    ws = wb[sheet_name]
    # 获取全部合并单元格坐标
    merged_coords = [mcr.coord for mcr in ws.merged_cells.ranges]
    data_start_row = 6
    supplier_col = 1
    week_start_col = 2
    template_styles = []
    for col in range(1, ws.max_column + 1):
        c = ws.cell(row=data_start_row, column=col)
        template_styles.append({"font": copy(c.font), "fill": copy(c.fill),
"border": copy(c.border),
                                "alignment": copy(c.alignment),
"number_format": c.number_format})
    for i, sid in enumerate(supplier_ids):
        r = data_start_row + i
        cell_s = ws.cell(row=r, column=supplier_col)
        # 如果当前单元格属于合并区域，找到左上角主单元格
        for coord_str in merged_coords:
            if cell_s.coordinate in coord_str:
                min_col, min_row, _, _ = range_boundaries(coord_str)
                cell_s = ws.cell(row=min_row, column=min_col)
                break
        cell_s.value = sid
        st = template_styles[supplier_col - 1]
        cell_s.font, cell_s.fill, cell_s.border, cell_s.alignment,
cell_s.number_format = st["font"], st["fill"], st["border"], st["alignment"],
st["number_format"]
    for widx in range(24):
        col = week_start_col + widx
        val = round(q_matrix[i, widx], 4)
        cell = ws.cell(row=r, column=col)
        stc = template_styles[col - 1]
        cell.font, cell.fill, cell.border, cell.alignment, cell.number_format

```

```

= stc["font"], stc["fill"], stc["border"], stc["alignment"], stc["number_format"]
        if abs(val) > 1e-6:
            cell.value = val
        else:
            cell.value = None
wb.save(template_path)
print(f'附件 A， 工作表[{sheet_name}]填充完成')

# =====填充附件 B=====
def fill_template_B(template_path, sheet_name, supplier_ids, transit_ids,
x_matrix, s_matrix):
    wb = load_workbook(template_path)
    ws = wb[sheet_name]
    merged_coords = [mcr.coord for mcr in ws.merged_cells.ranges]
    data_start_row = 6
    supplier_col = 1
    cols_per_week = n_transit
    template_styles = []
    for col in range(1, ws.max_column + 1):
        c = ws.cell(row=data_start_row, column=col)
        template_styles.append({"font": copy(c.font), "fill": copy(c.fill),
"border": copy(c.border),
                                "alignment": copy(c.alignment),
"number_format": c.number_format})
    for i, sid in enumerate(supplier_ids):
        r = data_start_row + i
        cell_s = ws.cell(row=r, column=supplier_col)
        for coord_str in merged_coords:
            if cell_s.coordinate in coord_str:
                min_col, min_row, _, _ = range_boundaries(coord_str)
                cell_s = ws.cell(row=min_row, column=min_col)
                break
        cell_s.value = sid
        st = template_styles[supplier_col - 1]
        cell_s.font, cell_s.fill, cell_s.border, cell_s.alignment,
cell_s.number_format = st["font"], st["fill"], st["border"], st["alignment"],
st["number_format"]

```

```

        for widx in range(24):
            week_start_col = 2 + widx * cols_per_week
            for j, _ in enumerate(transit_ids):
                col = week_start_col + j
                cell = ws.cell(row=r, column=col)
                stc = template_styles[col - 1]
                cell.font, cell.fill, cell.border, cell.alignment,
cell.number_format = stc["font"], stc["fill"], stc["border"], stc["alignment"],
stc["number_format"]
                if x_matrix[i, j, widx] == 1 and abs(s_matrix[i, widx]) > 1e-6:
                    val = round(s_matrix[i, widx], 4)
                    cell.value = val
                else:
                    cell.value = None

wb.save(template_path)
print(f'附件 B, 工作表[{sheet_name}]填充完成')
print("\n====正在填充附件 A、B====")
fill_template_A(TEMPLATE_A_PATH, SHEET_Q4, supplier_ids, q_matrix)
fill_template_B(TEMPLATE_B_PATH, SHEET_Q5, supplier_ids, transit_ids,
x_matrix, s_matrix)
print("\n 全部运行结束！ ")

```